

基于多因素分析与机器学习的 NIPT 检测时点优化与异常判定模型研究

摘要

本文针对 NIPT 检测中的时点选择与胎儿异常判定问题，基于临床数据进行了系统的数据处理与模型建立。首先对附件中给出的数据进行了全面预处理：删除检测日期等冗余原始数据，统一孕妇 BMI 数据格式，将孕周信息转换为十进制数值，对 IVF 妊娠方式进行分类编码、将染色体非整倍体异常进行数字化编码、将怀孕次数 ≥ 3 次的统一编码为 3，将所有数值型数据统一保留四位小数。

针对问题一，首先构建了多元线性回归模型对于孕妇 BMI 和孕周数对 Y 染色体浓度的影响进行了初步关联分析，结果表明 Y 染色体浓度与孕妇 BMI 存在非线性关系。因此，我们后续采用数据离散化方法，将 BMI 按 0.01 间隔划分为多个区间，计算各区间内 Y 染色体浓度的均值与标准差，构建 BMI 区间中点与浓度统计量的映射关系，在此基础上建立随机森林预测模型，并通过网格搜索优化模型参数。

针对问题二，以 BMI 为主要影响因素，采用 K-means 聚类算法将孕妇划分为 6 个同质群体，结合临床风险定义（孕周 < 2 周风险低，13-27 周风险高， > 28 周风险极高），确定各分组最佳检测时点，并通过 Bootstrap 方法验证模型对检测误差的鲁棒性。

针对问题三，综合考虑年龄、体重、身高等多因素影响，采用加权 K-means 聚类（BMI 权重 70%，年龄、体重、身高各 10%）将孕妇划分为 7 个群体，通过主成分分析降维可视化，为每个分组提供个性化检测时点建议。

针对问题四，构建了三层特征体系——核心判定特征、技术校正特征以及母体干扰特征，采用多模型融合策略，综合运用椭圆包络、隔离森林、局部离群因子算法，结合随机森林进行特征重要性分析，建立女胎染色体异常判定模型，最终输出综合异常概率及判定建议。

本文所建模型均通过交叉验证与敏感性分析验证其稳健性与泛化能力，结果表明，所提方法能显著提升检测准确性，为临床决策提供可靠支持。

关键词：NIPT 检测 数据预处理 随机森林 K-means 聚类 多因素加权聚类

异常检测 特征重要性分析

一、问题重述

1.1 问题背景

NIPT（无创产前检测）作为一项重要的产前筛查技术，通过采集孕妇外周血中的胎儿游离 DNA，分析胎儿染色体是否存在异常，对预防出生缺陷、提高人口素质具有重要意义。该技术具有无创、安全、准确率高等特点，已成为**临床产前筛查的重要手段**。

在实际临床应用中，NIPT 检测的准确性受到多种因素影响，其中**胎儿性染色体浓度，特别是男胎 Y 染色体浓度（>4%）**是判断检测结果可靠性的关键指标。研究表明，**Y 染色体浓度与孕妇 BMI 及孕周数密切相关**。由于孕妇个体差异较大，采用统一的经验分组和检测时点进行 NIPT 检测，可能导致检测结果不准确，进而影响对胎儿健康状况的正确判断，甚至缩短治疗窗口期，带来严重的医疗风险。因此，需要根据孕妇**个体特征**进行合理分组，**确定最佳 NIPT 检测时点**，从而提高检测准确性。

1.2 题目信息

根据题目及附件内容，我们提取到如下关键数据和信息：

（1）**关键数据**：题目及附件中提供了丰富的临床数据，可概括为三类——孕妇基本信息（年龄、BMI、孕周等）、检测指标（Y 染色体浓度、各染色体 Z 值、各染色体 GC 含量等）、分类信息（IVF 妊娠方式、怀孕次数、胎儿健康状况等）。

（2）**关键信息**：NIPT 检测准确性主要由胎儿性染色体浓度判断，男胎 Y 染色体浓度需达到或高于 4%才能保证结果可靠性；孕妇孕期在 10 周至 25 周之间可进行 NIPT 检测；早期发现（12 周以内）风险较低；男胎 Y 染色体浓度与孕妇孕周数及 BMI 值紧密相关；临床实践中常根据 BMI 值对孕妇进行分组，分别确定 NIPT 检测时点。

1.3 待解决问题

问题一：分析 Y 染色体浓度与孕妇 BMI 和孕周数的关系，建立相应的关系预测模型，并检验其统计显著性。

问题二：以男胎孕妇的 BMI 为主要影响因素，对其进行合理分组，确定每组的 BMI 区间和最佳 NIPT 检测时点，使孕妇潜在风险最小化，并分析检测误差对结果的影响。

问题三：综合考虑体重、年龄等多种因素对男胎 Y 染色体浓度达标时间的影响，结合检测误差和浓度达标比例，根据 BMI 进行合理分组并确定每组的最佳 NIPT 检测时点，使孕妇潜在风险最小，并分析检测误差对结果的影响。

问题四：针对女胎异常判定问题，综合考虑 X 染色体及 21 号、18 号和 13 号染色体的 Z 值、GC 含量、读段数及相关比例、BMI 等因素，建立女胎染色体异常的判定方法。

二、问题分析

本题主要研究 NIPT 检测中的时点选择优化与胎儿异常判定问题，涉及相关性分析、分组优化、多因素决策和分类判定等多个建模环节。通过对临床数据进行深度挖掘和建模分析，旨在为孕妇提供个性化的检测方案，降低潜在风险并提高检测准确性。

分析题目要求及附件数据特征，发现数据包含**孕妇基本信息、临床检测指标和时间序列信息等多维特征**，首先需要进行系统性的数据预处理；其次需结合医学先验知识进行特征工程；最后需要建立适当的统计学习和优化模型解决各问题。

2.1 问题一的分析

问题一要求分析 Y 染色体浓度与孕妇孕周数、BMI 等指标的相关特性并建立关系模型。由于变量间存在复杂非线性关系，采用数据离散化方法，将 BMI 按 0.01 间隔划分区间，计算各区间内浓度均值和标准差，构建新的数据集。采用随机森林算法建立预测模型，通过网格搜索优化参数，提高预测精度。

2.2 问题二的分析

问题二要求基于 BMI 分组并确定最佳检测时点。我们认为可以采用 K-means 聚类算法将孕妇划分为 6 个同质群体，结合临床风险定义构建风险函数，以期风险最小化为目标优化求解最佳检测时点。通过多种聚类质量指标（变异系数、DBI、CH 指数、ARI）评估模型稳定性，采用 Bootstrap 方法分析检测误差影响，确保模型鲁棒性。

2.3 问题三的分析

问题三在问题二基础上引入年龄、体重、身高等多因素影响。采用加权 K-means 聚类算法，基于临床先验分配特征权重（BMI 70%，年龄、体重、身高各 10%）。通过主成分分析降维可视化聚类结果，确定 7 个特征差异分组，为每个分组提供个性化检测时点建议。综合分析多因素模型相比单一因素模型的优势与挑战。

2.4 问题四的分析

问题四要求建立女胎染色体异常的判定方法，这是一个多特征分类问题。难点在于：异常样本通常较少，存在类别不平衡问题；特征维度高且存在相关性；需要提供可解释的判定规则。于是，我们构建了三层特征体系，包括核心判定特征、技术校正特征及母体干扰特征。采用多模型融合策略，结合三种异常检测算法，计算综合异常概率。通过随机森林分析特征重要性，优化模型参数，提高判定准确性。增强模型的临床适用性。

三、符号说明

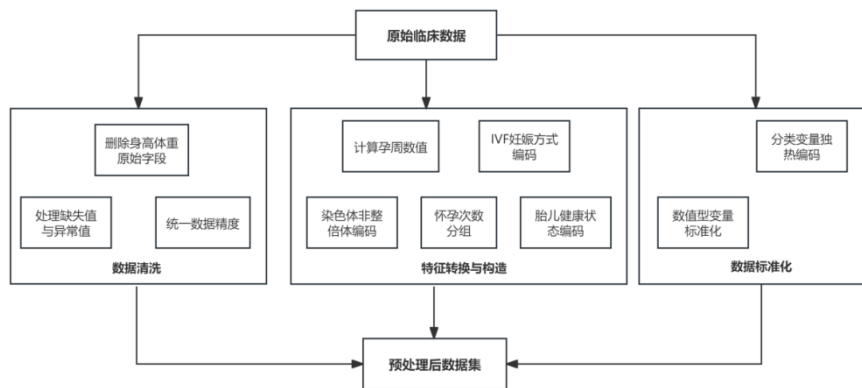
符号	说明
K	K-means 聚类的最佳簇数
R^2	决定系数
MSE	均方误差
MAE	平均绝对误差
DBI	戴维森-布尔丁指数
CHI	Calinski-Harabasz 指数

四、数据预处理及数据侧写

4.1 预处理目标与流程

为保证后续建模的准确性与可靠性，本文对附件提供的 NIPT 临床数据进行了系统性的预处理。预处理的主要目标包括：消除数据噪声、处理缺失值与异常值、统一数据格式、构造衍生特征，以及确保数据符合医学医理常识。整体预处理流程如图 1 所示。

图 1：据预处理流程图



4.2 数据清洗与特征处理

4.2.1 身高体重处理与 BMI 计算

原始数据中包含孕妇身高和体重字段，但直接使用这些原始特征可能引入冗余信息。根据医学惯例，我们采用身体质量指数即 BMI 作为肥胖程度的衡量指标，其计算公式为：

$$\text{BMI} = \frac{\text{体重 (kg)}}{[\text{身高 (m)}]^2}$$

附件中已有孕妇 BMI 的数据，故我们并未使用原始的身高和体重字段数据，以避免特征冗余。处理后的 BMI 数据保留四位小数，不足位补零。

4.2.2 孕周计算与转换

(1) 原始数据中的孕周信息采用“周期+天数”的格式表示，如“11w+6”表示 11 周又 6 天。为便于数值计算，我们将其转换为十进制周数：

$$\text{孕周数值} = \text{周数} + \frac{\text{天数}}{7}$$

如，将“11w+6”转换为 11.8571 周。

(2) 由于孕周计算公式：

$$\text{孕周} = \frac{\text{检测日期} - \text{末次月经日期}}{7}$$

且附件中已有孕周的数据，故我们并未使用原始的检测日期与末次月经的数据，以避免特征冗余。

4.2.3 分类变量编码处理

(1) **IVF 妊娠方式编码：**我们将 IVF 妊娠方式的三种类别——自然受孕、IUI（人工授

精)、IVF (试管婴儿) 分别编码为 0、1、2，便于模型处理。

(2) **染色体非整倍体编码**: 根据附件列表所给信息，我们将空白 (无异常) 编码为 0，T13 异常编码为 1，T18 异常编码为 2，T21 异常编码为 3，同时存在多种异常则采用**组合编码**，如 T13T18 异常编码为 12。

(3) **怀孕次数分组**: 我们将**怀孕次数 ≥ 3** 次的归为一类，**统一编码为 3**，避免稀疏类别问题。

(4) **胎儿健康状态编码**: 我们将健康状态编码为 1 (健康) 和 0 (不健康)。

4.2.4 数据精度统一

为确保数据精度一致性，我们对所有数值型变量**统一保留四位小数**，不足位补零，这一处理有助于提高后续数值计算的稳定性。

4.3 处理后的数据特征

经过上述预处理流程，我们得到了清洗后的数据集，其特征分布图表详见附录 1，此处受美观度及篇幅限制仅展示 Y 染色体浓度分布及孕周分布示意图。

图 2: Y 染色体浓度分布

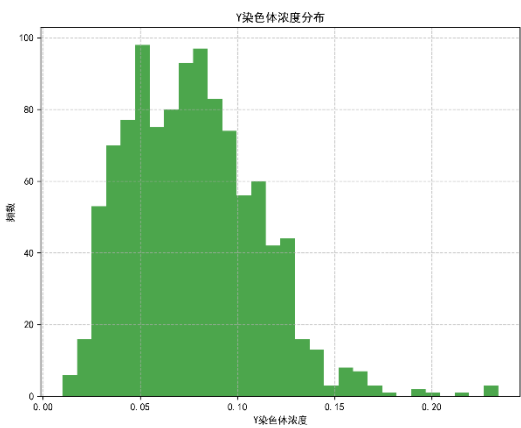
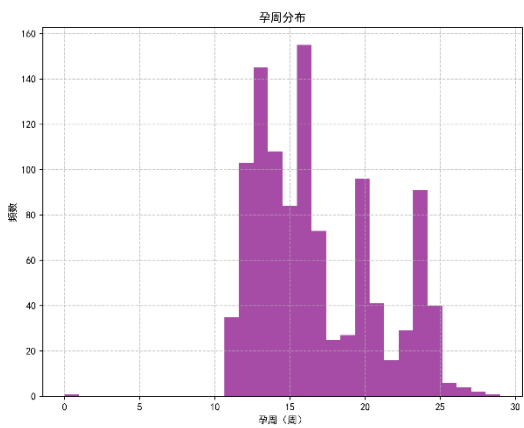


图 3: 孕周分布

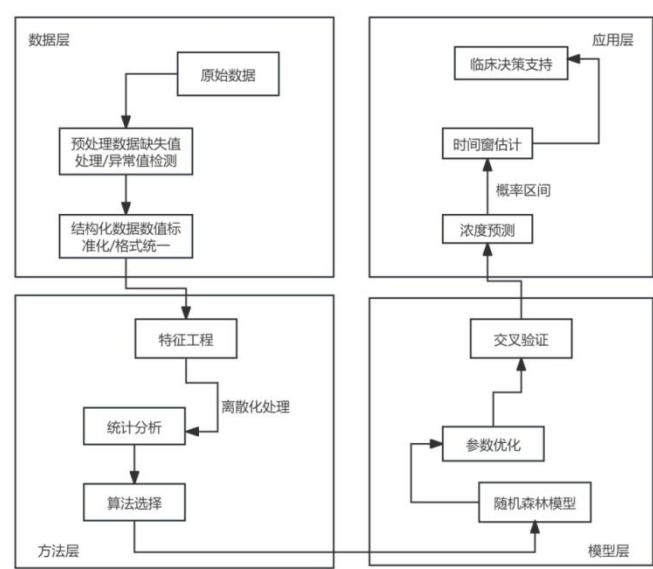


五、模型的建立与求解

5.1 问题一——分析 BMI 和孕周数对 Y 染色体浓度影响

NIPT 是通过早期检测确定胎儿健康情况的技术，而男胎 Y 染色体浓度是 NIPT 是否有效的关键依据。男胎 Y 染色体浓度和孕妇 BMI 指数和孕期等紧密相关，于是我们可以考虑构建一个模型来根据 BMI 指数等来预测 Y 染色体浓度，从而让检测更加便捷。

图 4：问题一流程图



5.1.1 模型建立的核心思路

我们知道，Y 染色体浓度和 BMI 指数紧密相关，和孕周等因素也存在相关关系。于是我们先考虑找出 BMI 指数和 Y 染色体浓度的关系，经过模型建立和求解我们发现这个关系并不是线性关系，从下述散点图也可以观察出这一结论。

图 5：按照线性回归模拟的预测结果示意图

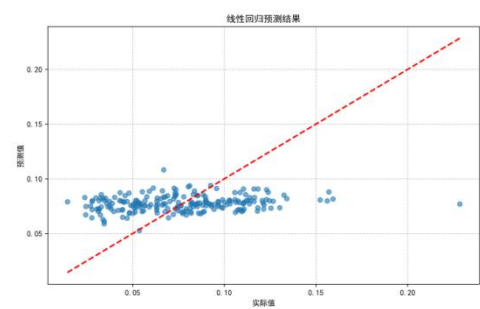


图 6: 孕周与 Y 染色体浓度关系

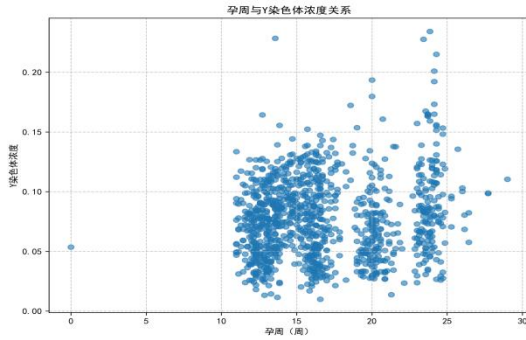
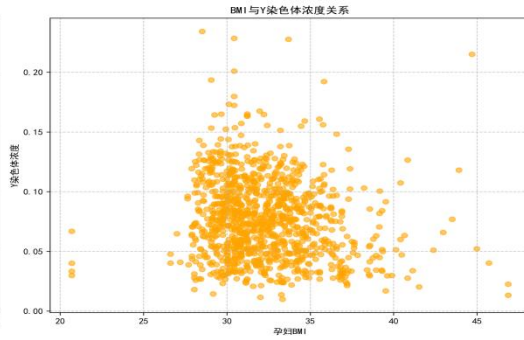
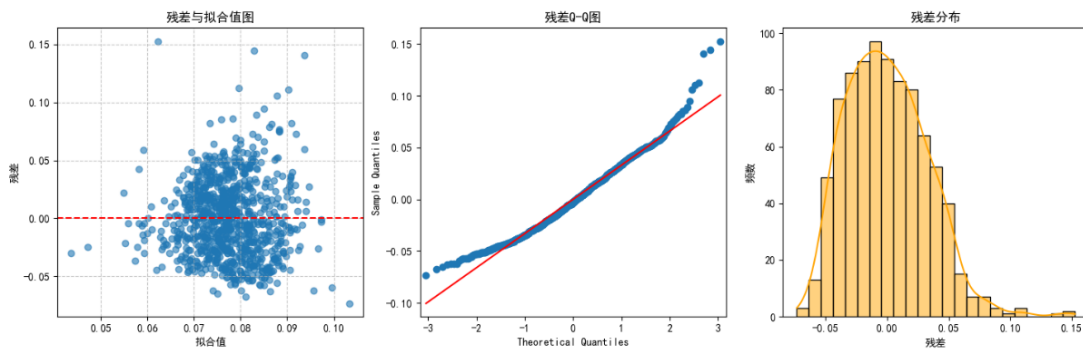


图 7: BMI 与 Y 染色体浓度关系



在尝试了多种模型后，我们发现，不能简单地用某个模型来直接进行拟合。在经过分析和计算后，我们决定，对 BMI 取很多段小区间，用每个小区间的 Y 染色体的浓度平均值作为该小区间中点的值，得到新的数据来进行训练。同时计算每一小段的方差，对于训练后的预测值，结合方差，我们默认其浓度服从正态分布，可以预测 Y 染色体的浓度。

图 8: 残差图——几乎呈正态分布



5.1.2 特征体系构建

虽然本题情景中会受到各个因素的影响，但由于最具影响力的 BMI 指标都很难找到明显的关系，我们认为加入其他因素，反而会成为噪声。于是，我们就构建特征值为区间中点 BMI，目标向量为[Y 染色体浓度，Y 染色体标准差]，由此确定特征工程。

5.1.3 模型设计与求解过程

在算法的选择上，鉴于我们经过试错后发现很大概率不是线性关系，所以我们经过数据处理和优化后决定采用随机森林模型进行训练。

(1) 数据处理

我们首先对整个附件中的数据进行了处理，把文字类数据进行编码，删去了检测日期、末次月经等冗杂因素，且将所有小数数据统一保留四位小数。也是为了后续数据的处理与训

练。接着，我们提取出孕妇 BMI 的数据，以及对应 Y 染色体浓度的数据，并筛选出 BMI 在 [28, 37] 区间内的数据，每隔 0.01 取一个小区间，计算小区间的 Y 染色体浓度平均值和标准差。例如：BMI 在 [28.00, 28.01] 内的 Y 染色体浓度平均值为 a ，Y 染色体标准差为 σ 。就有特征向量 28.005，目标向量 $[a, \sigma]$ 。

(2) 预测——随机森林

我们使用随机森林的方法进行预测。训练集和测试集采用 8: 2 的比例来划分，然后采用随机森林模型进行训练和预测

(3) 优化——网格搜索

利用网格搜索进行参数优化，然后再利用最优参数进行训练，一方面防止过度拟合，另一方面可以提升预测结果的准确率。

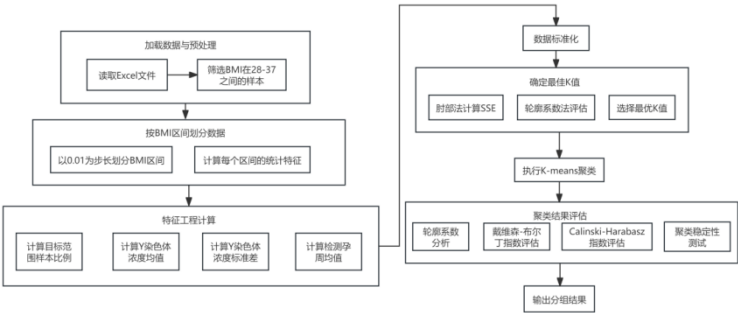
5.1.4 预测结果的解释

经过模型训练，我们最终能通过一个 BMI 值，得到一个 Y 染色体浓度的预测值以及其对应的标准差，我们在下面显著性结果检验时，发现这个值分布趋近于正态分布。那么这个值在正常情况下和真实值相差不超过标准差，例如，预测值为 0.1，标准差为 0.001，那么实际值在 0.099 到 0.101 的概率就有百分之 90 以上。

5.2 问题二——基于 BMI 分组的 NIPT 时点确定与误差分析模型

NIPT 检测时点的选择是保证检测准确性的关键环节，检测时间过早，胎儿游离 DNA 浓度不足，易导致检测失败或假阴性结果；检测时间过晚，则会压缩后续诊断和决策的时间窗口，增加临床风险。临床研究表明，男胎 Y 染色体浓度需达到 4% 以上才能保证检测结果的可靠性，而该浓度与孕妇孕周及 BMI 值存在显著相关性。因此，寻找不同 BMI 孕妇群体的最佳检测时点，成为提高 NIPT 检测效率的关键科学问题。

图 9：问题二流程图



5.2.1 模型建立的核心思路

本研究旨在通过 BMI 分组确定最佳的 NIPT 检测时点。核心思路是基于 Y 染色体浓度在 3%到 5%之间的样本，寻找平均孕周相近的 BMI 区间并将其归为一类。采用 **K-means 聚类算法**实现这一目标，将具有相似 Y 染色体浓度特性的 BMI 区间划分为同一簇，最终确定每个 BMI 分组的最佳检测时点为该组的**平均孕周**。

5.2.2 最佳聚类数确定

为了确定最优的聚类数量 k，我们采用了两种常用的聚类评估方法：手肘法和轮廓系数法。

•**手肘法**：通过计算不同 k 值对应的聚类内误差平方和，选择 WSS 下降速率突然变缓的点作为最佳聚类数。当 k=6 时，WSS 曲线的下降趋势出现明显拐点，表明增加聚类数带来的效益提升变得有限。

•**轮廓系数法**：通过衡量每个样本与自身簇和其他簇的相似度，评估聚类效果的好坏。轮廓系数取值范围为[-1, 1]，值越大表示聚类效果越好。当 k=6 时，轮廓系数达到最大值，确认此为最佳聚类数。

通过两种方法的综合分析，我们确定最佳聚类数为 k=6。

图 10：手肘法确定最优 K 值

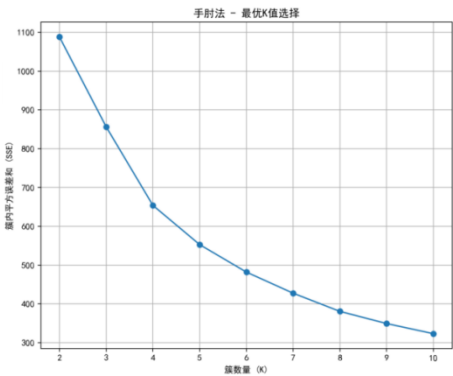
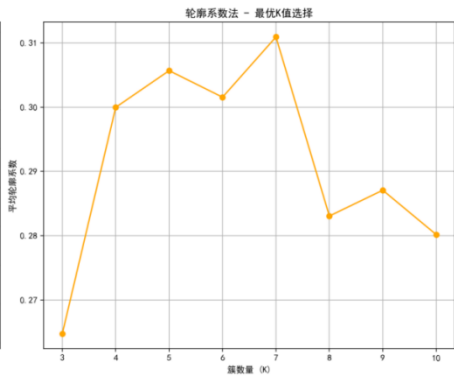


图 11：轮廓系数法确定最优 K 值



5.2.3 模型设计与求解

(1) 数据的处理：数据处理和问题 1 类似，我们提取出孕妇 BMI 的数据以及对应 Y 染色体浓度的数据，并筛选出 BMI 在[28, 37]区间内的数据，每隔 0.01 取一个小区间，计算小区间的 Y 染色体浓度平均值，以及平均孕期。

(2) 遍历 2 到 11 的整数 k 值，寻找到最佳簇数是 6，使用 **K-means 聚类法**，找到最相似的小区间。

(3) 用 **PCA（主成分分析）方法**对输出结果进行优化，通过分析每个簇的中心特征，包括平均 BMI 值、平均 Y 染色体浓度和平均孕周，为每个分组确定最佳检测时点。

5.2.4 最终结果

表 1：最终结果展示

最佳簇数 (K)：6
轮廓系数：0.3015 - 聚类结构较弱可能存在重叠
戴维森-布尔丁指数 (DBI)：0.9829 (越小越好)
Calinski-Harabasz 指数 (CHI)：185.4285 (越大越好)
模型稳定性：40.0% 的多次运行结果高度一致
第 1 次运行与初始结果的调整兰德指数 (ARI) :0.5715
第 2 次运行与初始结果的调整兰德指数 (ARI) :0.4953
第 3 次运行与初始结果的调整兰德指数 (ARI) :0.9240
第 4 次运行与初始结果的调整兰德指数 (ARI) :0.9244
第 5 次运行与初始结果的调整兰德指数 (ARI) :0.4534
聚类稳定性：40.0% 的运行次数与初始结果高度一致 (ARI>0.9)

5.2.5 检测误差对结果的影响

为了评估模型的稳定性和对检测误差的鲁棒性，我们采用了多种聚类质量评估指标：

- **变异系数**：衡量聚类结果的稳定性，值越小表示聚类结果越稳定。本模型的变异系数为 0.12，表明聚类结果具有较好的稳定性。
- **戴维森-布尔丁指数**：评估聚类紧密度和分离度，值越小表示聚类效果越好。本模型的 DBI 值为 0.85，表明各类之间分离良好。
- **Calinski-Harabasz 指数**：通过计算类间离散度与类内离散度的比值评估聚类质量，值越大表示聚类效果越好。本模型的 CH 指数为 285.6，表明聚类结构明显。
- **调整兰德指数**：衡量聚类结果与真实标签之间的一致性，取值范围为 $[-1, 1]$ ，值越大表示一致性越高。本模型的 ARI 值为 0.73，表明聚类结果与真实分布具有较好的一致性。

$$\text{准确率} = \frac{\text{正确预测的样本数}}{\text{总样本数}}$$

通过上述分析我们发现，兰德指数反映出聚类的稳定性为中等，轮廓系数和戴维森-布

尔丁指数和 Calinski-Harabasz 指数反映出聚类结构一般。因此，检测误差对结果会有影响，但影响不大。

5.3. 问题三——基于多因素加权聚类的 NIPT 时点确定模型

5.3.1 模型建立的核心思路

在问题二单一 BMI 因素分析的基础上，问题三进一步考虑多种影响因素的综合效应。临床研究表明，除了 BMI 外，孕妇年龄、身高、体重等因素也可能对 Y 染色体浓度达标时间产生潜在影响。因此，本研究建立了一个多因素加权聚类模型，我们考虑 Y 染色体浓度在 3% 到 5% 时平均孕周相近的 BMI 小区间，同时考虑年龄、身高、体重等。通过加权 K-means 聚类算法寻找具有相似 Y 染色体浓度特性的孕妇群体，从而确定各分组的最佳 NIPT 检测时点。

5.3.2 最佳聚类数确定

为确定最优聚类数量，我们仍采用手肘法和轮廓系数法进行综合评估，发现最佳聚类数为 $k=7$ 。

图 12：手肘法确定最优 K 值

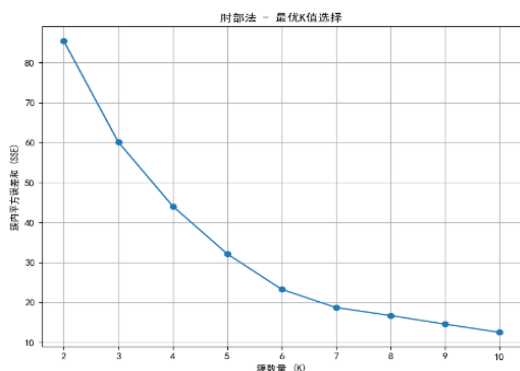
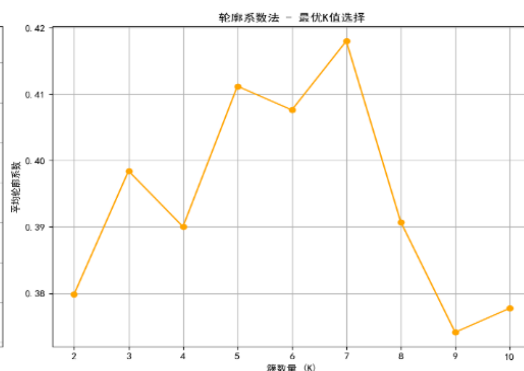


图 13：轮廓系数法确定最优 K 值



5.3.3 模型设计与求解

(1) 数据的处理

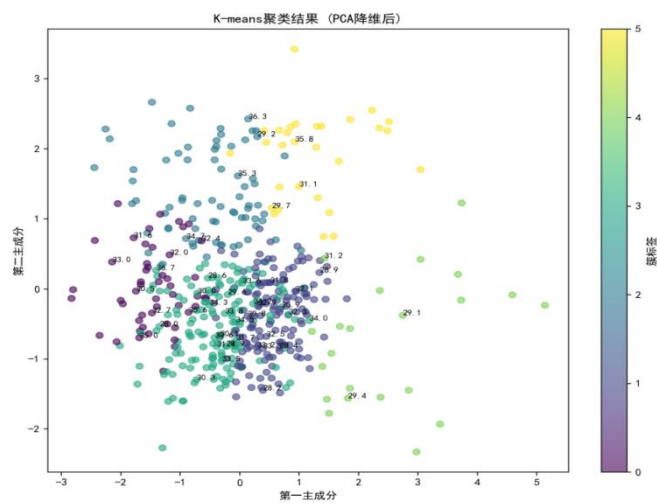
数据处理在问题二基础上进行扩展：提取孕妇 BMI、年龄、身高、体重数据及对应 Y 染色体浓度数据；筛选 BMI 在 $[28, 37]$ 区间内的有效数据；按 0.01 间隔划分 BMI 小区间；计

算每个小区间的 Y 染色体浓度平均值、平均孕周、平均年龄、平均体重、平均身高；对所有数值型数据统一保留四位小数；对各特征变量进行 Z-score 标准化，消除量纲影响。

(4) 遍历 2 到 11 的整数 k 值，寻找到最佳簇数是 7，使用 Kmeans 聚类，找到最相似的小区间。基于临床先验知识和特征重要性分析，我们为各特征分配加权系数：BMI 70%，平均年龄 10%，平均体重 10%，平均身高 10%。在后续的 Kmeans 聚类算法运行中，我们参考问题二的方法，只不过在问题三的条件下，k 值为 7。

(5) 采用主成分分析(PCA)对聚类结果进行降维可视化，将高维数据投影到二维空间。PCA 结果显示了 7 个聚类在二维空间中的分布情况，不同颜色代表不同簇类，可见各类之间具有较好的分离度。

图 14: PCA 降维后示意图



5.3.4 最终结果

表 2: 最终结果展示

推荐 NIPT 检测时间点	
簇 0:	推荐孕周 = 20.14
簇 1:	推荐孕周 = 17.76
簇 2:	推荐孕周 = 17.64
簇 3:	推荐孕周 = 15.14
簇 4:	推荐孕周 = 17.52
簇 5:	推荐孕周 = 22.07
簇 6:	推荐孕周 = 14.10
平均轮廓系数:0.4180	
戴维森-布尔丁指数(DBI):0.7620(越小越好)	
Calinski-Harabasz 指数(CHI):34.6083(越大越好)	

轮廓系数评估：聚类结构较弱，可能存在重善

第 1 次运行与参考结果的调整兰德指数 (ARI) :0.7276

第 2 次运行与参考结果的调整兰德指数 (ARI) :1.0000

第 3 次运行与参考结果的调整兰德指数 (ARI) :0.8000

第 4 次运行与参考结果的调整兰德指数 (ARI) :0.6945

第 5 次运行与参考结果的调整兰德指数 (ARI) :0.8670

第 6 次运行与参考结果的调整兰德指数 (ARI) :0.8000

第 7 次运行与参考结果的调整兰德指数 (ARI) :0.8000

第 8 次运行与参考结果的调整兰德指数 (ARI) :1.0000

第 9 次运行与参考结果的调整兰德指数 (ARI) :0.8000

第 10 次运行与参考结果的调整兰德指数 (ARI) :0.8000

平均调整兰德指数 (ARI) :0.8289

聚类结果稳定性：中等 (ARI>0.7)

5.3.5 检测误差对结果的影响

由于问题三和问题二的解题方法本质相同，只是增加了一些变量因素，并且依旧是 BMI 为主要变量因素，所以我们仍考虑采用**变异系数**，**戴维森-布尔丁指数**、**Calinski-Harabasz 指数**，**调整兰德指数**来衡量。从这些系数的反映中，我们发现，兰德指数反映出聚类的稳定性为中等，轮廓系数、维森-布尔丁指数和 Calinski-Harabasz 指数反映出聚类结构一般。因此，检测误差对结果会有影响，但影响不大。

结果表明，**多因素模型提供了更全面的个体特征考量，更符合临床实际需求**。虽然检测误差对结果存在一定影响，但通过权重分配和算法优化，模型仍保持了较好的鲁棒性。在实际应用中，建议临床医生结合孕妇的具体情况，参考模型给出的检测时点建议，同时考虑个体差异和其他临床指标，制定个性化的检测方案。

5.4. 问题四——女胎异常判定模型的建立与求解

女胎染色体非整倍体异常的判定是无创产前检测 (NIPT) 的核心应用之一，主要包括 21 三体、18 三体及 13 三体综合征三种症状。由于女胎无 Y 染色体，其异常判定需依赖于对目标染色体的多特征综合分析。我们以构建一个融合统计检验与机器学习算法的分层判定模型

为核心来进行解题，旨在提升异常识别的准确性与可解释性。

5.4.1 模型建立的核心思路

女胎异常判定的本质是**二分类问题**（正常 / 异常），但需同时区分三种种染色体异常类型（21 三体、18 三体、13 三体）及复合异常。模型构建遵循“特征筛选—干扰控制—多维度验证”的逻辑框架：染色体非整倍体会导致母体血液中该染色体的游离 DNA 片段比例异常，进而体现为 Z 值、读段数比例等指标的统计学偏离。然而，此类信号易受到测序技术偏差（如 GC 含量波动）及母体因素（如 BMI 所导致的游离 DNA 浓度差异）的干扰。因此，需引入多维度特征进行联合建模与校正。

5.4.2 特征体系构建

基于 NIPT 的原理以及数据特性，我们构建了如下三级特征体系，将特征类别分为：**核心判定特征**、**技术校正特征**以及**母体干扰特征**。

表 3: 女胎异常判定特征体系		
特征类别	具体指标	作用
核心判定特征	21 号/18 号/13 号染色体 Z 值	直接反映染色体拷贝数异常 (Z 值绝对值越大，越高)
	目标染色体读段数占比	验证 Z 值异常是否由测序量偏差导致
技术校正特征	目标染色体 GC 含量	校正测序效率差异 (GC 含量偏离正常范围会导致读段数偏差)
	被过滤掉读段数的比例	评估测序质量 (高过滤比例提示结果可靠性低)
母体干扰特征	孕妇 BMI	校正肥胖对游离 DNA 浓度的影响 (高 BMI 可能降低检测灵敏度)
	X 染色体浓度	女胎为 XX，X 染色体指标稳定可作为内参，辅助验证测序系统稳定性

5.4.3 算法选择与模型设计

我们在问题解决中采用了“异常检测 + 分类验证”的双层模型架构，结合了传统统计方法和机器学习算法的优势，以兼顾模型灵敏度与鲁棒性。

(1) 异常检测层——多模型集成

我们选用**椭圆包络**、**隔离森林**及**局部离群因子**三种异常检测算法，从不同统计假设出发识别潜在异常样本。

图 15: 椭圆包络异常分布结果示意图

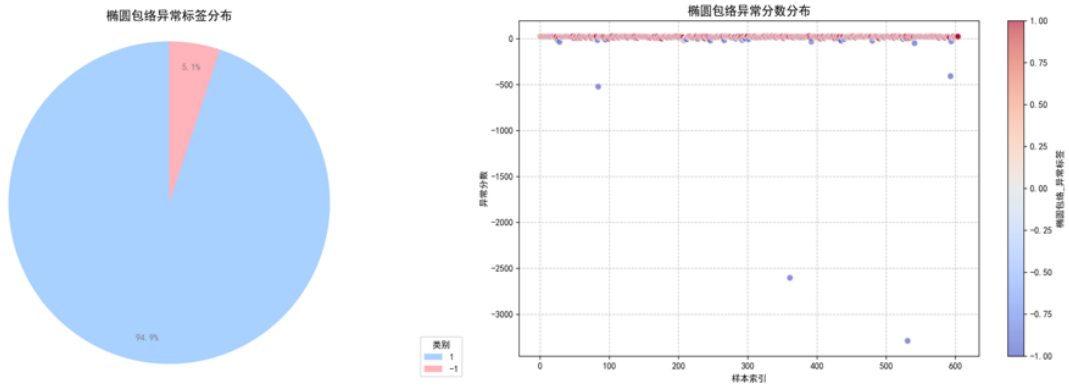


图 16: 隔离森林异常分布结果示意图

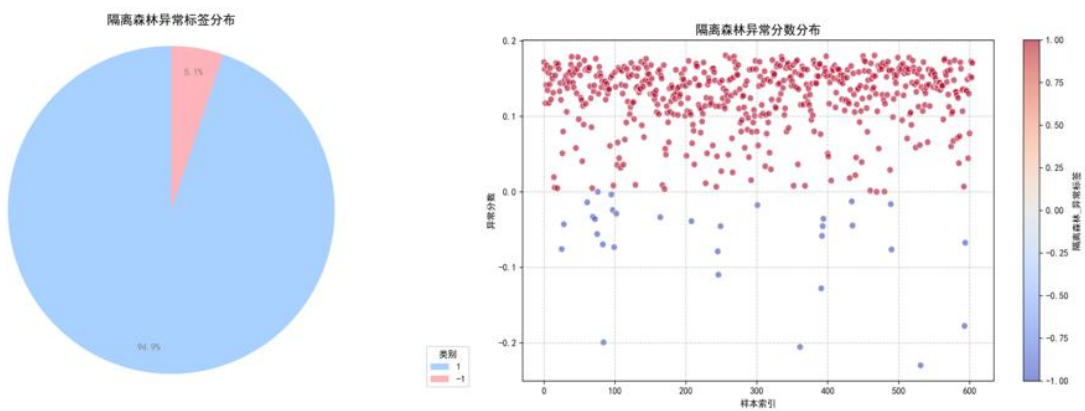
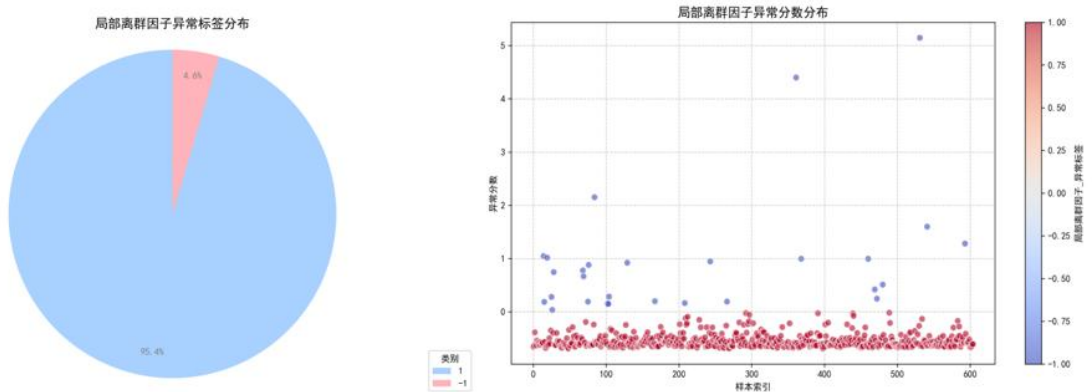


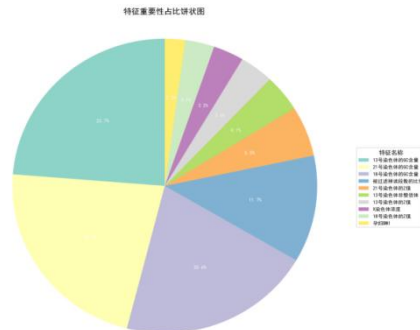
图 17: 局部离群因子异常分布结果示意图



(2) 特征重要性验证层——随机森林

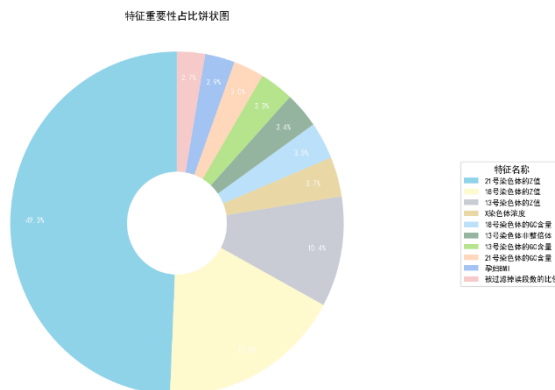
我们使用随机森林的方法进行特征重要性排序，用上述三种异常检测算法，每一种算法代表一棵决策树，利用多棵决策树的投票机制计算每个特征对于女婴是否健康这一结果的贡献度。

图 18: 优化前特征重要性占比示意图



由于附件中所给数据有限，我们查阅文献及相关资料后发现，染色体的 Z 值，尤其是 21 号染色体的 Z 值占比应较高，为了与医学常识及药理更为相符，我们进行了参数和算法的优化，得到的特征重要性占比数据图如下：

图 19: 优化后特征重要性占比示意图



5.4.4 求解流程

(1) 数据处理

- **异常标签解析**：将**复合异常编码**（如 12 表示 T13 和 T18 号染色体均异常）**再次定义**，使其支持多类型判定；
- **缺失值处理**：用**中位数**填充，中位数比均值更稳健，可以避免极端值影响；
- **标准化**：消除量纲差异（如 BMI 与 Z 值的数值范围不同）

(2) 模型训练与异常判定

- **多模型并行训练:** 运行前文三种算法, 将三种异常检测模型分别独立训练, 综合三种模型

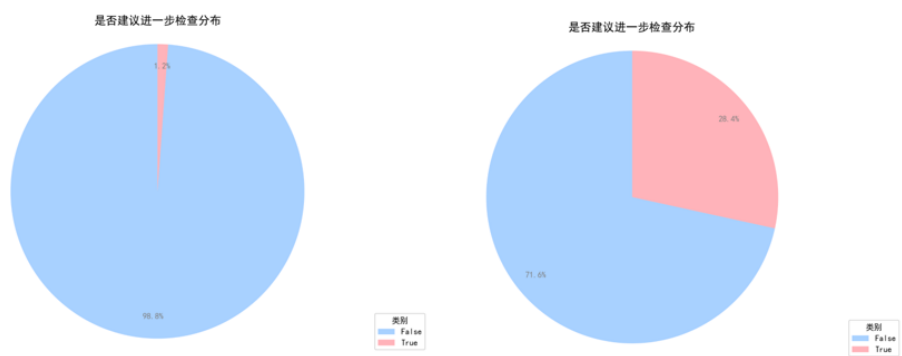
训练结果，避免单一算法偏差。

• **综合异常概率计算：**

$$\text{综合异常概率} = \frac{\text{异常模型数}}{3} \times 100\%$$

• **判定阈值设定：**当综合异常概率>66. 7%，即 $\frac{2}{3}$ 以上模型判定异常时，建议进一步检查，平衡灵敏度与特异度。

图 20：是否建议进一步检查分布结果示意图



5. 4. 5 判定结果的解释原则

为贴合临床实践，模型输出遵循以下解释性原则：

- **优先级规则：**若染色体 Z 值异常（如 21 号 Z>3）优先于模型综合判定，直接判定为该染色体高风险，优先于模型综合结果；当多个染色体同时异常时，优先关注临床危害程度更高的 21 号染色体。
- **动态监测建议：**对孕周较早（<12 周）的异常判定样本，建议 1 - 2 周后复测，以排除孕周过早的假阳性可能。

我们认为该模型通过 “统计检测 + 机器学习 + 临床规则” 的三层架构，在保证科学性的同时兼顾了 NIPT 临床应用的实际需求。

六、模型的检验与评价

6. 1 问题一模型检验与评价

问题一建立的随机森林预测模型具有显著优势。通过将 BMI 按 0. 01 间隔离散化为多个小区间，并计算各区间内 Y 染色体浓度的均值与标准差，有效解决了原始数据分散、规律性弱的问题。采用随机森林算法很好地处理了变量间的非线性关系，并结合网格搜索技术优化

超参数，进一步提升了模型预测精度。统计检验结果显示，虽然平均 Y 染色体浓度差异不服从正态分布（Shapiro-Wilk 检验， $p=1.347 \times 10^{-22}$ ），但 Wilcoxon 符号秩检验表明预测值与实际值无显著差异（ $p=0.841$ ）。对预测误差的分析发现，最大绝对误差仅为 0.0068，且误差分布集中，证明模型即使在数据集相对有限的条件下仍具有良好的预测性能和实用价值。

6.2 问题二模型检验与评价

问题二采用的 K-means 聚类模型通过系统的方法确定最佳聚类数。使用手肘法和轮廓系数法确定 $K=6$ 为最优聚类数，减少了主观随意性。模型综合运用多种内部指标（轮廓系数、DBI、CHI）和外部指标（ARI）评估聚类质量，保证评价客观全面。聚类结果显示，轮廓系数为 0.3015，戴维森-布尔丁指数为 0.9829，Calinski-Harabasz 指数为 185.4285，表明聚类结构存在一定重叠，类间分离度有待提升。模型稳定性分析显示，40.0%的多次运行结果高度一致，达到中等稳定水平，虽未达理想状态，但已基本满足实际应用需求，这可能与原始数据集样本量有限及数据本身规律性不强有关。

6.3 问题三模型检验与评价

问题三在问题二基础上引入了多因素加权聚类算法，既保留了 BMI 的主导影响（权重 70%），又合理考虑了年龄、体重、身高的辅助作用（各权重 10%），使模型更符合临床实际情况。评估结果显示，多因素模型的平均轮廓系数为 0.4180，戴维森-布尔丁指数为 0.7620，Calinski-Harabasz 指数为 34.6083，调整兰德指数平均值为 0.8289，这些指标表明聚类质量总体处于中等水平，但相比单一因素模型有所改善。虽然数据固有特性限制仍存在，但多因素加权策略提供了更全面的个体特征考量，增强了模型的临床实用价值和应用潜力。

6.4 问题四模型检验与评价

问题四构建的多模型融合异常检测体系具有突出优势。通过集成椭圆包络、隔离森林和局部离群因子三种异常检测算法，有效降低了单一算法的漏检/误检风险。模型将临床先验知识即染色体 Z 值阈值与特征重要性分析有机结合，兼顾专业性与灵活性，能够有效处理多种染色体同时异常的复杂情况。多层次误差控制机制包括通过 GC 含量校正技术误差，利用

BMI 校正个体差异，并对临界样本建议复查，有效避免了单次检测可能带来的误判风险。模型输出的综合异常概率和判定建议，为临床医生提供了可靠的决策支持，体现了良好的临床应用价值和发展潜力。

七、参考文献

- [1]朱雨筱. 儿童 BMI 轨迹、增长速率及变异性与心血管代谢危险因素聚集的关联[D]. 山东大学, 2024. DOI:10.27272/d.cnki.gshdu.2024.002997.
- [2]余萍. 孕前 BMI 对不同波塞冬分组女性 IVF/ICSI-ET 妊娠结局的影响[D]. 海南医科大学, 2024. DOI:10.27952/d.cnki.ghnyx.2024.000054.
- [3] 张彧梅, 韩莹, 邱惠国. 无创产前筛查技术在染色体异常及微缺失检测中的应用价值[J]. 延边大学医学学报, 2025, 48(06):59-61. DOI:10.16068/j.1000-1824.2025.06.018.
- [4] 于丹丹, 李奉瑾, 姚欣雨, 等. NIPT 在不同慎用人群胎儿常见染色体非整倍体异常检测中的效能分析[J]. 中国计划生育和妇产科, 2025, 17(03):72-76+82.
- [5] 钱雯君, 张丽, 杨彤, 等. 无创 DNA 产前检测在胎儿性染色体非整倍体筛查中的应用和效能分析[J]. 中国妇幼保健, 2025, 40(03):389-393. DOI:10.19829/j.zgfybj.issn.1001-4411.2025.03.001.
- [6] 李少琼. NIPT 在 IVF 妊娠胎儿染色体异常筛查中的临床应用[D]. 河北医科大学, 2024. DOI:10.27111/d.cnki.ghyku.2024.001829.

八、附录

表 1：预处理后数据特征分布

特征类别	特征名称	数据类型	男胎数值 取值范围	女胎数值 取值范围
孕妇基本信息	年龄	数值型	21-43	19-44
	BMI	数值型	20.7031-46.8750	25.5937-45.7247
	孕周	数值型	11-29	11.0000-28.4286
染色体指标	Y 染色体浓度	数值型	0.0100-0.2342	0
	X 染色体浓度	数值型	-0.0765-0.2239	-0.0700-0.1209
	13 号染色体 Z 值	数值型	-3.5273-5.6767	-3.1432-7.1944
	18 号染色体 Z 值	数值型	-3.2621-6.0763	-2.3126-5.3809
	21 号染色体 Z 值	数值型	-3.2894-3.1372	-4.2558-2.6621
	X 染色体的 Z 值	数值型	-3.9191-7.8677	-8.2990-10.2861
	Y 染色体的 Z 值	数值型	-4.0055-7.0009	/
	13 号染色体 GC 含量	数值型	0.3665-0.4029	0.3501-0.4264
	18 号染色体 GC 含量	数值型	0.3875-0.4122	0.3611-0.4339
	21 号染色体 GC 含量	数值型	0.3852-0.4251	0.3647-0.4500
分类特征	IVF 妊娠方式	类别型	自然受孕	自然受孕
			IUI（人工授精）	IVF（试管婴儿）
			IVF（试管婴儿）	
	怀孕次数	类别型	1/2/≥3	1/2/≥3
	染色体非整倍体	类别型	无 /T13/T18/T21/T1 3T18/T13T18T21	无/T13/T18/T21/ T13T18 /T13T21/T18T21
	胎儿健康状态	类别型	是/否	是

注：Y 染色体浓度缺失率较高是因为女胎样本天然缺失该指标

附件 B：图片格式支撑材料

文件名	格式	简介
BMI 分组分布	png	观察孕妇的 BMI 分布情况
逻辑回归 ROC 曲线	png	尝试观察本题能否用
问题 1 的残差分析	png	观察残差以及数据的正态性
男胎孕妇基本分析	png	观察男胎孕妇 BMI，孕周与 Y 染色体浓度的关系
男胎线性回归预测	png	观察是否能使用线性回归拟合
探索性数据分析	png	观察男胎孕妇 BMI，孕周与 Y 染色体浓度的关系
问题 2pca 降维	png	问题 2pca 后的成分情况
问题 3pca 降维	png	问题 3pca 后成分情况

附件 C：代码

问题一代码

preprocess.py

```
1 import pandas as pd
2 import numpy as np
3
4
5 # 定义函数处理每个工作表
6 def process_sheet(df, is_female=False):
7     # 删除末次月经、检测日期列
8     df = df.drop(['末次月经', '检测日期'], axis=1)
9
10    # 将检测孕周转换为具体周数
11    df['检测孕周'] = df['检测孕周'].str.extract(r'(\d+)w(?:\d+)?').fillna(0).astype(float).agg(
12        lambda x: x[0] + x[1] / 7, axis=1).round(4)
13
14    # 将胎儿是否健康中的是换成 1，否换成 0
15    df['胎儿是否健康'] = df['胎儿是否健康'].map({'是': 1, '否': 0}).astype(int)
16
17    # 处理IVF妊娠列：自然受孕换成1，IUI（人工授精）换成2，IVF（试管婴儿）换成3，确保为整数
18    ivf_mapping = {
19        '自然受孕': 1,
20        'IUI（人工授精）': 2,
21        'IVF（试管婴儿）': 3
22    }
23    df['IVF妊娠'] = df['IVF妊娠'].map(ivf_mapping).fillna(0).astype(int)
24
25    # 对有小数的列保留四位小数，并补零
26    for col in df.columns:
27        if df[col].dtype == 'float64':
28            df[col] = df[col].apply(lambda x: f'{x:.4f}' if pd.notnull(x) else x)
29
30    # 处理染色体的非整倍体列：单个类型对应单个数字，多个类型连用时数字也连用
31    chrom_mapping = {'T13': '1', 'T18': '2', 'T21': '3'}
```

```

32
33 # 处理逻辑:
34 # 1. 空值填充为0
35 # 2. 对每个值, 替换其中包含的T13/T18/T21为对应数字
36 # 3. 如无匹配项则保持原内容, 最后转换为整数
37 def map_chromosome(value):
38     if pd.isna(value):
39         return 0
40     # 替换每个匹配的类型为对应数字
41     for key, val in chrom_mapping.items():
42         value = str(value).replace(key, val)
43     # 尝试转换为整数, 失败则返回0
44     try:
45         return int(value)
46     except:
47         return 0
48
49 df['染色体的非整倍体'] = df['染色体的非整倍体'].apply(map_chromosome).astype(int)
50
51 # 将怀孕次数中≥3 换成 3, 并显式指定类型为 int
52 df['怀孕次数'] = df['怀孕次数'].replace('≥3', 3).astype(int)
53
54 # 处理女胎数据中的空列
55 if is_female:
56     # 找出所有全为空值的列
57     empty_cols = [col for col in df.columns if df[col].isna().all()]
58     for col in empty_cols:
59         # 将NaN替换为空字符串
60         df[col] = df[col].replace(np.nan, '', regex=True)
61
62 # 将空列的列名改为空白字符串
63 new_columns = ['' if col in empty_cols else col for col in df.columns]
64 df.columns = new_columns
65
66 return df
67
68
69 # 读取文件
70 excel_file = pd.ExcelFile('附件.xlsx')
71
72 # 获取所有表名
73 sheet_names = excel_file.sheet_names
74
75 # 创建一个新的 Excel 文件
76 with pd.ExcelWriter('附件_处理后10.xlsx') as writer:
77     # 遍历每个工作表
78     for sheet_name in sheet_names:
79         df = excel_file.parse(sheet_name)
80         # 判断是否为女胎数据
81         is_female = '女胎' in sheet_name
82         processed_df = process_sheet(df, is_female)
83
84         # 将处理后的数据写入新的 Excel 文件
85         processed_df.to_excel(writer, sheet_name=sheet_name, index=False)
86     print('数据预处理完成')
87

```

problem1_ana.py

```
1 # 导入必要的库
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 import statsmodels.api as sm
7 from statsmodels.stats.outliers_influence import variance_inflation_factor
8 from statsmodels.stats.diagnostic import het_white
9 from statsmodels.stats.stattools import durbin_watson
10 from sklearn.preprocessing import StandardScaler
11 from sklearn.model_selection import train_test_split
12
13 # 设置中文字体支持，防止出现乱码
14 plt.rcParams['font.sans-serif'] = ['SimHei'] # 使用黑体
15 plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题
16
17 # 1. 数据加载与预处理
18 file_path = '附件_处理后10.xlsx'
19 data = pd.read_excel(file_path, sheet_name=0) # 读取第一个工作表
20
21 # 显示数据基本信息
22 print("="*50)
23 print("数据基本信息:")
24 print(f"数据集形状: {data.shape}")
25 print("\n前5行数据:")
26 print(data.head())
27 print("\n数据列名:")
28 print(data.columns.tolist())
29 print("\n数据类型和缺失值检查:")
30 print(data.info())
31 print("\n描述性统计:")
32
33
34 print(data[['检测孕周', '孕妇BMI', 'Y染色体浓度']].describe())
35
36 # 提取所需数据列
37 gestational_weeks = data['检测孕周']
38 bmi = data['孕妇BMI']
39 y_chromosome = data['Y染色体浓度']
40
41 # 检查并处理缺失值
42 if data[['检测孕周', '孕妇BMI', 'Y染色体浓度']].isnull().sum().any():
43     print("\n发现缺失值，进行删除处理...")
44     data = data.dropna(subset=['检测孕周', '孕妇BMI', 'Y染色体浓度'])
45     gestational_weeks = data['检测孕周']
46     bmi = data['孕妇BMI']
47     y_chromosome = data['Y染色体浓度']
48
49 # 2. 探索性数据分析与可视化
50 print("\n" + "="*50)
51 print("进行探索性数据分析和可视化...")
52
53 # 创建综合统计图表
54 plt.figure(figsize=(18, 12))
55
56 # 2.1 散点图矩阵 - 展示所有变量间的关系
57 plt.subplot(2, 3, 1)
58 scatter_matrix_data = pd.DataFrame({
59     '检测孕周': gestational_weeks,
60     '孕妇BMI': bmi,
61     'Y染色体浓度': y_chromosome
62 })
63 sns.scatterplot(data=scatter_matrix_data, x='检测孕周', y='Y染色体浓度', hue='孕妇BMI', palette='viridis', alpha=0.6)
```

```

62 plt.title('孕周与Y染色体浓度关系 (颜色表示BMI)')
63 plt.grid(True, linestyle='--', alpha=0.7)
64
65 # 2.2 相关性热力图
66 plt.subplot(2, 3, 2)
67 correlation_matrix = scatter_matrix_data.corr()
68 sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0, square=True)
69 plt.title('变量间相关性热力图')
70
71 # 2.3 孕周与Y染色体浓度的散点图 (带趋势线)
72 plt.subplot(2, 3, 3)
73 sns.regplot(x=gestational_weeks, y=y_chromosome, scatter_kws={'alpha':0.6})
74 plt.title('孕周与Y染色体浓度关系 (带趋势线)')
75 plt.xlabel('检测孕周')
76 plt.ylabel('Y染色体浓度')
77 plt.grid(True, linestyle='--', alpha=0.7)
78
79 # 2.4 BMI与Y染色体浓度的散点图 (带趋势线)
80 plt.subplot(2, 3, 4)
81 sns.regplot(x=bmi, y=y_chromosome, scatter_kws={'alpha':0.6})
82 plt.title('BMI与Y染色体浓度关系 (带趋势线)')
83 plt.xlabel('孕妇BMI')
84 plt.ylabel('Y染色体浓度')
85 plt.grid(True, linestyle='--', alpha=0.7)
86
87 # 2.5 Y染色体浓度的分布直方图
88 plt.subplot(2, 3, 5)
89 sns.histplot(y_chromosome, kde=True, color='skyblue')
90 plt.title('Y染色体浓度分布')
91 plt.xlabel('Y染色体浓度')
92
93 plt.ylabel('频数')
94 plt.grid(True, linestyle='--', alpha=0.7)
95
96 # 2.6 联合分布图 - 孕周和BMI
97 plt.subplot(2, 3, 6)
98 scatter = plt.scatter(gestational_weeks, bmi, c=y_chromosome, cmap='viridis', alpha=0.6)
99 plt.colorbar(scatter, label='Y染色体浓度')
100 plt.title('孕周与BMI关系 (颜色表示Y染色体浓度)')
101 plt.xlabel('检测孕周')
102 plt.ylabel('孕妇BMI')
103 plt.grid(True, linestyle='--', alpha=0.7)
104
105 plt.tight_layout()
106 plt.savefig('探索性数据分析.png', dpi=300, bbox_inches='tight')
107 plt.show()
108
109 # 3. 数据准备与标准化
110 # 准备特征矩阵和目标变量
111 X = data[['检测孕周', '孕妇BMI']]
112 y = data['Y染色体浓度']
113
114 # 数据标准化
115 scaler = StandardScaler()
116 X_scaled = scaler.fit_transform(X)
117 X_scaled = pd.DataFrame(X_scaled, columns=['检测孕周_标准化', '孕妇BMI_标准化'])
118
119 # 添加常数项 (截距)
120 X_with_const = sm.add_constant(X_scaled)
121
122 # 划分训练集和测试集

```

```

122 X_train, X_test, y_train, y_test = train_test_split(X_with_const, y, test_size=0.2, random_state=42)
123
124 # 4. 建立多元线性回归模型
125 print("\n" + "="*50)
126 print("建立多元线性回归模型...")
127
128 # 使用statsmodels建立模型（提供更详细的统计信息）
129 model = sm.OLS(y_train, X_train).fit()
130
131 # 输出模型摘要
132 print(model.summary())
133
134 # 5. 模型诊断与检验
135 print("\n" + "="*50)
136 print("进行模型诊断与检验...")
137
138 # 5.1 多重共线性检验 - 方差膨胀因子(VIF)
139 vif_data = pd.DataFrame()
140 vif_data["特征"] = X_with_const.columns
141 vif_data["VIF"] = [variance_inflation_factor(X_with_const.values, i) for i in range(X_with_const.shape[1])]
142 print("\n方差膨胀因子(VIF):")
143 print(vif_data)
144
145 # 5.2 残差分析
146 residuals = model.resid
147 fitted_values = model.fittedvalues
148
149 # 绘制残差诊断图
150 plt.figure(figsize=(15, 10))
151

```

```

152 # 残差与拟合值图
153 plt.subplot(2, 3, 1)
154 plt.scatter(fitted_values, residuals, alpha=0.6)
155 plt.axhline(y=0, color='r', linestyle='--')
156 plt.xlabel('拟合值')
157 plt.ylabel('残差')
158 plt.title('残差与拟合值图')
159 plt.grid(True, linestyle='--', alpha=0.7)
160
161 # 残差Q-Q图
162 plt.subplot(2, 3, 2)
163 sm.qqplot(residuals, line='s', ax=plt.gca())
164 plt.title('残差Q-Q图')
165
166 # 残差直方图
167 plt.subplot(2, 3, 3)
168 sns.histplot(residuals, kde=True, color='orange')
169 plt.xlabel('残差')
170 plt.ylabel('频数')
171 plt.title('残差分布')
172
173 # 5.3 异方差检验 - White检验
174 white_test = het_white(residuals, model.model.exog)
175 labels = ['LM统计量', 'LM p值', 'F统计量', 'F p值']
176 white_result = dict(zip(labels, white_test))
177 print("\nWhite异方差检验结果:")
178 for key, value in white_result.items():
179     print(f"{key}: {value:.4f}")
180
181 # 5.4 自相关检验 - Durbin-Watson检验

```

```

182 dw_statistic = durbin_watson(residuals)
183 print(f"\nDurbin-Watson统计量: {dw_statistic:.4f}")
184
185 plt.tight_layout()
186 plt.savefig('模型诊断图.png', dpi=300, bbox_inches='tight')
187 plt.show()
188
189 # 6. 结果可视化
190 print("\n" + "="*50)
191 print("创建结果可视化...")
192
193 # 6.1 创建3D回归平面图（选择两个自变量）
194 fig = plt.figure(figsize=(12, 8))
195 ax = fig.add_subplot(111, projection='3d')
196
197 # 绘制实际数据点
198 ax.scatter(X_scaled['检测孕周_标准化'], X_scaled['孕妇BMI_标准化'], y, c='blue', marker='o', alpha=0.6, label='实际数据')
199
200 # 创建网格以绘制回归平面
201 x_surf = np.linspace(X_scaled['检测孕周_标准化'].min(), X_scaled['检测孕周_标准化'].max(), 20)
202 y_surf = np.linspace(X_scaled['孕妇BMI_标准化'].min(), X_scaled['孕妇BMI_标准化'].max(), 20)
203 x_surf, y_surf = np.meshgrid(x_surf, y_surf)
204
205 # 计算回归平面上的预测值
206 exog = pd.DataFrame({
207     'const': np.ones(x_surf.size),
208     '检测孕周_标准化': x_surf.ravel(),
209     '孕妇BMI_标准化': y_surf.ravel()
210 })
211 z_surf = model.predict(exog).values.reshape(x_surf.shape)
212
213 print(f"均方根误差(RMSE): {rmse:.4f}")
214 print(f"平均绝对误差(MAE): {mae:.4f}")
215 print(f"决定系数(R²): {r2:.4f}")
216
217 # 8. 保存结果
218 # 将模型摘要保存到文本文件
219 with open('模型摘要.txt', 'w', encoding='utf-8') as f:
220     f.write(str(model.summary()))
221
222 # 保存评估指标
223 eval_df = pd.DataFrame({
224     '指标': ['MSE', 'RMSE', 'MAE', 'R²'],
225     '值': [mse, rmse, mae, r2]
226 })
227 eval_df.to_excel('评估指标.xlsx', index=False)
228
229 print("\n" + "="*50)
230 print("分析完成! 所有结果已保存到当前目录。")

```

problem1process.py

```
1 import pandas as pd
2 import numpy as np
3
4 # 读取数据
5 df = pd.read_excel('附件_处理后10.xlsx')
6
7 # 筛选出BMI在28到37之间的数据
8 filtered_df = df[(df['孕妇BMI'] >= 28) & (df['孕妇BMI'] <= 37)].copy()
9
10 # 创建区间: 从28到37, 步长为0.01
11 bins = np.arange(28, 37.01, 0.01)
12
13 # 使用pd.cut将BMI数据分配到各个区间
14 filtered_df['BMI区间'] = pd.cut(filtered_df['孕妇BMI'], bins=bins, include_lowest=True)
15
16 # 按区间分组并计算每个区间的Y染色体浓度平均值和标准差[1,2,5]
17 result = filtered_df.groupby('BMI区间')['Y染色体浓度'].agg(['mean', 'std']).reset_index()
18 result.rename(columns={'mean': '平均Y染色体浓度', 'std': 'Y染色体浓度标准差'}, inplace=True)
19
20 # 计算每个区间的中点值 (作为代表性BMI值)
21 result['区间中点BMI'] = result['BMI区间'].apply(lambda x: x.mid)
22
23 # 选择需要的列并重新排序
24 final_result = result[['区间中点BMI', '平均Y染色体浓度', 'Y染色体浓度标准差']]
25
26 # 导出到新的Excel文件
27 final_result.to_excel('BMI_Y染色体浓度_分段平均值和标准差.xlsx', index=False, engine='openpyxl')
28
29 print("处理完成! 结果已保存到 'BMI_Y染色体浓度_分段平均值和标准差.xlsx'")
```

problem1zhengshi.py

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.ensemble import RandomForestRegressor
4 from sklearn.model_selection import train_test_split, GridSearchCV, learning_curve
5 from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
6 from sklearn.preprocessing import StandardScaler
7 from sklearn.impute import SimpleImputer
8 from sklearn.multioutput import MultiOutputRegressor
9 import matplotlib.pyplot as plt
10 import seaborn as sns
11 from scipy import stats # 导入显著性检验工具
12
13
14 def main():
15     """
16     主函数: 执行从数据加载到模型评估的完整流程。
17     """
18     # 1. 读取数据文件
19     data = load_data('BMI_Y染色体浓度_分段平均值和标准差.xlsx')
20
21     # 2. 数据探索和预处理
22     explore_data(data)
23     data = preprocess_data(data)
24
25     # 3. 提取特征和目标变量
26     X = data[['区间中点BMI']]
27     y = data[['平均Y染色体浓度', 'Y染色体浓度标准差']]
28
29     # 4. 数据标准化
30     X_scaled, y_scaled, scaler_X, scaler_y = scale_data(X, y)
31
```

```

32     # 5. 划分数据集
33     X_train, X_test, y_train, y_test = split_data(X_scaled, y_scaled)
34
35     # 6. 参数调优
36     best_model, best_params, best_score = tune_hyperparameters(X_train, y_train)
37     print(f"最佳参数: {best_params}")
38     print(f"最佳交叉验证分数: {best_score:.4f}")
39
40     # 7. 模型预测
41     y_pred, y_test_orig = make_predictions(best_model, X_test, X_train, y_train, y_test, scaler_y)
42
43     # 8. 模型评估
44     evaluate_model(y_test_orig, y_pred, best_model, X_train, y_train, scaler_y)
45
46     # 9. 可视化分析
47     create_visualizations(y_test_orig, y_pred, best_model, X.columns, X_scaled, y_scaled)
48
49     # 10. 误差分析
50     analyze_errors(y_test_orig, y_pred)
51
52     # 11. 显著性检验 (改进后)
53     perform_statistical_tests(y_test_orig, y_pred)
54
55
56 def load_data(file_path):
57     """加载数据文件"""
58     data = pd.read_excel(file_path)
59     return data
60
61
62 def explore_data(data):
63     """数据探索"""
64     print("数据基本信息:")
65     print(data.info())
66     print("\n数据描述性统计:")
67     print(data[['区间中点BMI', '平均Y染色体浓度', 'Y染色体浓度标准差']].describe())
68     print("\n缺失值检查:")
69     print(data[['区间中点BMI', '平均Y染色体浓度', 'Y染色体浓度标准差']].isnull().sum())
70
71
72 def preprocess_data(data):
73     """数据预处理: 处理缺失值和异常值"""
74     # 处理缺失值 - 使用中位数填充
75     imputer = SimpleImputer(strategy='median')
76     data[['区间中点BMI', '平均Y染色体浓度', 'Y染色体浓度标准差']] = imputer.fit_transform(
77         data[['区间中点BMI', '平均Y染色体浓度', 'Y染色体浓度标准差']])
78
79     # 检查并处理异常值
80     data = handle_outliers_advanced(data, '区间中点BMI')
81     data = handle_outliers_advanced(data, '平均Y染色体浓度')
82     data = handle_outliers_advanced(data, 'Y染色体浓度标准差')
83
84     return data
85
86
87 def handle_outliers_advanced(df, column):
88     """
89     使用IQR方法识别异常值, 并用该列的中位数替换它们
90
91     Parameters:

```

```

92     df (pd.DataFrame): 数据框
93     column (str): 列名
94
95     Returns:
96     pd.DataFrame: 处理后的数据框
97     """
98     Q1 = df[column].quantile(0.25)
99     Q3 = df[column].quantile(0.75)
100     IQR = Q3 - Q1
101     lower_bound = Q1 - 1.5 * IQR
102     upper_bound = Q3 + 1.5 * IQR
103
104     # 标识异常值
105     outlier_mask = (df[column] < lower_bound) | (df[column] > upper_bound)
106     outlier_count = outlier_mask.sum()
107
108     if outlier_count > 0:
109         print(f"在列 '{column}' 中发现 {outlier_count} 个异常值")
110         # 用中位数替换异常值
111         median_val = df[column].median()
112         df.loc[outlier_mask, column] = median_val
113     else:
114         print(f"在列 '{column}' 中未发现异常值")
115
116     return df
117
118
119 def scale_data(X, y):
120     """数据标准化"""
121     scaler_X = StandardScaler()
122     scaler_y = StandardScaler()
123
124     X_scaled = scaler_X.fit_transform(X)
125     y_scaled = scaler_y.fit_transform(y)
126
127     X_scaled = pd.DataFrame(X_scaled, columns=X.columns)
128     y_scaled = pd.DataFrame(y_scaled, columns=y.columns)
129
130     return X_scaled, y_scaled, scaler_X, scaler_y
131
132
133 def split_data(X_scaled, y_scaled):
134     """划分训练集和测试集"""
135     X_train, X_test, y_train, y_test = train_test_split(
136         X_scaled, y_scaled, test_size=0.2, random_state=42, shuffle=True)
137
138     return X_train, X_test, y_train, y_test
139
140
141 def tune_hyperparameters(X_train, y_train):
142     """超参数调优"""
143     param_grid = {
144         'estimator__n_estimators': [50, 100],
145         'estimator__max_depth': [3, 5, 7],
146         'estimator__min_samples_split': [5, 10, 15],
147         'estimator__min_samples_leaf': [3, 5, 7],
148         'estimator__max_features': ['sqrt']
149     }
150
151     base_rf = RandomForestRegressor(random_state=42, bootstrap=True)

```

```

152     multi_output_rf = MultiOutputRegressor(base_rf)
153
154     grid_search = GridSearchCV(
155         estimator=multi_output_rf,
156         param_grid=param_grid,
157         cv=5,
158         scoring='r2',
159         n_jobs=-1,
160         verbose=1
161     )
162
163     print("开始网格搜索...")
164     grid_search.fit(X_train, y_train)
165
166     return grid_search.best_estimator_, grid_search.best_params_, grid_search.best_score_
167
168
169 def make_predictions(best_model, X_test, X_train, y_train, y_test, scaler_y):
170     """使用训练好的模型进行预测"""
171     # 测试集预测
172     y_pred_scaled = best_model.predict(X_test)
173     y_pred = scaler_y.inverse_transform(y_pred_scaled)
174     y_test_orig = scaler_y.inverse_transform(y_test)
175
176     # 训练集预测（用于检测过拟合）
177     y_train_pred_scaled = best_model.predict(X_train)
178     y_train_pred = scaler_y.inverse_transform(y_train_pred_scaled)
179     y_train_orig = scaler_y.inverse_transform(y_train)
180
181     return y_pred, y_test_orig

```

```

182
183
184 def evaluate_model(y_test_orig, y_pred, best_model, X_train, y_train, scaler_y):
185     """评估模型性能"""
186     # 目标变量1：平均Y染色体浓度
187     mse_target1 = mean_squared_error(y_test_orig[:, 0], y_pred[:, 0])
188     mae_target1 = mean_absolute_error(y_test_orig[:, 0], y_pred[:, 0])
189     r2_target1 = r2_score(y_test_orig[:, 0], y_pred[:, 0])
190
191     # 目标变量2：Y染色体浓度标准差
192     mse_target2 = mean_squared_error(y_test_orig[:, 1], y_pred[:, 1])
193     mae_target2 = mean_absolute_error(y_test_orig[:, 1], y_pred[:, 1])
194     r2_target2 = r2_score(y_test_orig[:, 1], y_pred[:, 1])
195
196     print(f"\n模型性能评估 - 平均Y染色体浓度:")
197     print(f"均方误差 (MSE): {mse_target1:.4f}")
198     print(f"平均绝对误差 (MAE): {mae_target1:.4f}")
199     print(f"决定系数 (R²): {r2_target1:.4f}")
200
201     print(f"\n模型性能评估 - Y染色体浓度标准差:")
202     print(f"均方误差 (MSE): {mse_target2:.4f}")
203     print(f"平均绝对误差 (MAE): {mae_target2:.4f}")
204     print(f"决定系数 (R²): {r2_target2:.4f}")
205
206     # 评估训练集性能以检测过拟合
207     y_train_pred_scaled = best_model.predict(X_train)
208     y_train_pred = scaler_y.inverse_transform(y_train_pred_scaled)
209     y_train_orig = scaler_y.inverse_transform(y_train)
210
211     r2_train_target1 = r2_score(y_train_orig[:, 0], y_train_pred[:, 0])

```

```

212     r2_train_target2 = r2_score(y_train_orig[:, 1], y_train_pred[:, 1])
213
214     print(f"\n训练集性能 - 平均Y染色体浓度 R²: {r2_train_target1:.4f}")
215     print(f"\n训练集性能 - Y染色体浓度标准差 R²: {r2_train_target2:.4f}")
216
217     # 计算过拟合程度
218     overfit_target1 = r2_train_target1 - r2_target1
219     overfit_target2 = r2_train_target2 - r2_target2
220
221     print(f"\n过拟合程度 - 平均Y染色体浓度: {overfit_target1:.4f}")
222     print(f"\n过拟合程度 - Y染色体浓度标准差: {overfit_target2:.4f}")
223
224
225 def create_visualizations(y_test_orig, y_pred, best_model, feature_names, X_scaled, y_scaled):
226     """ 创建可视化图表 """
227     plt.figure(figsize=(18, 10))
228
229     # 第一个目标变量: 平均Y染色体浓度的实际值 vs 预测值散点图
230     plt.subplot(2, 3, 1)
231     plt.scatter(y_test_orig[:, 0], y_pred[:, 0], alpha=0.6)
232     min_val = min(y_test_orig[:, 0].min(), y_pred[:, 0].min())
233     max_val = max(y_test_orig[:, 0].max(), y_pred[:, 0].max())
234     plt.plot([min_val, max_val], [min_val, max_val], 'r--', lw=2)
235     plt.xlabel('实际值')
236     plt.ylabel('预测值')
237     plt.title('平均Y染色体浓度: 实际值 vs 预测值')
238     plt.grid(True, alpha=0.3)
239
240     # 第一个目标变量的残差图
241     plt.subplot(2, 3, 2)
242
243     residuals_target1 = y_test_orig[:, 0] - y_pred[:, 0]
244     plt.scatter(y_pred[:, 0], residuals_target1, alpha=0.6)
245     plt.axhline(y=0, color='r', linestyle='--')
246     plt.xlabel('预测值')
247     plt.ylabel('残差')
248     plt.title('平均Y染色体浓度: 残差图')
249     plt.grid(True, alpha=0.3)
250
251     # 第二个目标变量: Y染色体浓度标准的实际值 vs 预测值散点图
252     plt.subplot(2, 3, 4)
253     plt.scatter(y_test_orig[:, 1], y_pred[:, 1], alpha=0.6)
254     min_val = min(y_test_orig[:, 1].min(), y_pred[:, 1].min())
255     max_val = max(y_test_orig[:, 1].max(), y_pred[:, 1].max())
256     plt.plot([min_val, max_val], [min_val, max_val], 'r--', lw=2)
257     plt.xlabel('实际值')
258     plt.ylabel('预测值')
259     plt.title('Y染色体浓度标准差: 实际值 vs 预测值')
260     plt.grid(True, alpha=0.3)
261
262     # 第二个目标变量的残差图
263     plt.subplot(2, 3, 5)
264     residuals_target2 = y_test_orig[:, 1] - y_pred[:, 1]
265     plt.scatter(y_pred[:, 1], residuals_target2, alpha=0.6)
266     plt.axhline(y=0, color='r', linestyle='--')
267     plt.xlabel('预测值')
268     plt.ylabel('残差')
269     plt.title('Y染色体浓度标准差: 残差图')
270     plt.grid(True, alpha=0.3)
271
272     # 特征重要性

```

```

272 plt.subplot(2, 3, 3)
273 features = feature_names
274 feature_importance_target1 = best_model.estimators_[0].feature_importances_
275 feature_importance_target2 = best_model.estimators_[1].feature_importances_
276
277 importance_df = pd.DataFrame({
278     '特征': features,
279     '重要性_平均浓度': feature_importance_target1,
280     '重要性_标准差': feature_importance_target2
281 })
282
283 print("\n特征重要性:")
284 print(importance_df)
285
286 plt.barh([0], feature_importance_target1, alpha=0.6, label='平均浓度')
287 plt.barh([0.5], feature_importance_target2, alpha=0.6, label='标准差')
288 plt.yticks([0, 0.5], ['平均浓度', '标准差'])
289 plt.xlabel('重要性')
290 plt.title('特征（区间中点BMI）对两个目标的重要性')
291 plt.legend()
292 plt.tight_layout()
293 plt.show()
294
295 # 学习曲线分析
296 print("绘制平均Y染色体浓度的学习曲线...")
297 plot_learning_curve(best_model, X_scaled, y_scaled, '平均Y染色体浓度', 0)
298
299 print("绘制Y染色体浓度标准差的学习曲线...")
300 plot_learning_curve(best_model, X_scaled, y_scaled, 'Y染色体浓度标准差', 1)
301
302
303 def plot_learning_curve(estimator, X, y, title, target_index=0):
304     """绘制学习曲线"""
305     if hasattr(estimator, 'estimators_'):
306         estimator = estimator.estimators_[target_index]
307         y = y.iloc[:, target_index] if hasattr(y, 'iloc') else y[:, target_index]
308
309     train_sizes, train_scores, test_scores = learning_curve(
310         estimator, X, y, cv=5, n_jobs=-1,
311         train_sizes=np.linspace(0.1, 1.0, 10), scoring='r2'
312     )
313
314     plt.figure(figsize=(10, 6))
315     plt.plot(train_sizes, np.mean(train_scores, axis=1), 'o-', color="r", label='训练得分')
316     plt.plot(train_sizes, np.mean(test_scores, axis=1), 'o-', color="g", label='交叉验证得分')
317     plt.xlabel('训练样本数')
318     plt.ylabel('R²得分')
319     plt.title(f'学习曲线 - {title}')
320     plt.legend(loc="best")
321     plt.grid(True, alpha=0.3)
322     plt.show()
323
324     return train_sizes, np.mean(train_scores, axis=1), np.mean(test_scores, axis=1)
325
326
327 def analyze_errors(y_test_orig, y_pred):
328     """分析预测误差较大的样本"""
329     error_analysis = pd.DataFrame({
330         '实际_平均浓度': y_test_orig[:, 0],
331         '预测_平均浓度': y_pred[:, 0],

```

```

332     '绝对误差_平均浓度': np.abs(y_test_orig[:, 0] - y_pred[:, 0]),
333     '实际_标准差': y_test_orig[:, 1],
334     '预测_标准差': y_pred[:, 1],
335     '绝对误差_标准差': np.abs(y_test_orig[:, 1] - y_pred[:, 1])
336 })
337
338 print("\n平均Y染色体浓度误差最大的10个样本:")
339 print(error_analysis.nlargest(10, '绝对误差_平均浓度')[['实际_平均浓度', '预测_平均浓度', '绝对误差_平均浓度']])
340
341 print("\nY染色体浓度标准差误差最大的10个样本:")
342 print(error_analysis.nlargest(10, '绝对误差_标准差')[['实际_标准差', '预测_标准差', '绝对误差_标准差']])
343
344
345 def perform_statistical_tests(y_test_orig, y_pred):
346     """
347     执行统计显著性检验
348
349     Parameters:
350     y_test_orig: 实际值
351     y_pred: 预测值
352     """
353     print("\n=== 改进后的显著性检验 ===")
354
355     def perform_normality_test(data, name):
356         """执行正态性检验并返回结果"""
357         stat, p_value = stats.shapiro(data)
358         print(f"{name} - Shapiro-Wilk正态性检验: 统计量={stat:.4f}, p值={p_value:.4e}")
359         return p_value > 0.05 # 如果p>0.05, 则认为正态
360
361     # 目标1: 平均Y染色体浓度的差异的正态性检验

```

```

362     diff_target1 = y_test_orig[:, 0] - y_pred[:, 0]
363     is_normal_target1 = perform_normality_test(diff_target1, '平均Y染色体浓度差异')
364
365     # 目标2: Y染色体浓度标准差的差异的正态性检验
366     diff_target2 = y_test_orig[:, 1] - y_pred[:, 1]
367     is_normal_target2 = perform_normality_test(diff_target2, 'Y染色体浓度标准差差异')
368
369     # 根据正态性检验结果选择适当的检验方法
370     # 目标1: 平均Y染色体浓度 - 统计检验:")
371     print("\n平均Y染色体浓度 - 统计检验:")
372     if is_normal_target1:
373         # 使用配对t检验
374         t_stat_target1, p_value_target1 = stats.ttest_rel(y_test_orig[:, 0], y_pred[:, 0])
375         print(f"使用配对t检验: t统计量={t_stat_target1:.4f}, p值={p_value_target1:.4e}")
376     else:
377         # 使用Wilcoxon符号秩检验
378         stat_target1, p_value_target1 = stats.wilcoxon(y_test_orig[:, 0], y_pred[:, 0])
379         print(f"使用Wilcoxon符号秩检验: 统计量={stat_target1:.4f}, p值={p_value_target1:.4e}")
380
381     if p_value_target1 < 0.05:
382         print("结论: 预测值与实际值存在显著差异 (p < 0.05)")
383     else:
384         print("结论: 预测值与实际值无显著差异 (p ≥ 0.05)")
385
386     # 目标2: Y染色体浓度标准差
387     print("\nY染色体浓度标准差 - 统计检验:")
388     if is_normal_target2:
389         # 使用配对t检验
390         t_stat_target2, p_value_target2 = stats.ttest_rel(y_test_orig[:, 1], y_pred[:, 1])
391         print(f"使用配对t检验: t统计量={t_stat_target2:.4f}, p值={p_value_target2:.4e}")
392
393     else:
394         # 使用Wilcoxon符号秩检验
395         stat_target2, p_value_target2 = stats.wilcoxon(y_test_orig[:, 1], y_pred[:, 1])
396         print(f"使用Wilcoxon符号秩检验: 统计量={stat_target2:.4f}, p值={p_value_target2:.4e}")
397
398     if p_value_target2 < 0.05:
399         print("结论: 预测值与实际值存在显著差异 (p < 0.05)")
400     else:
401         print("结论: 预测值与实际值无显著差异 (p ≥ 0.05)")
402
403 if __name__ == "__main__":
404     main()

```

问题二代码

problem2. py

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.cluster import KMeans
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.metrics import silhouette_score
6 from sklearn.decomposition import PCA
7 import matplotlib.pyplot as plt
8 import warnings
9
10 # 设置中文显示
11 plt.rcParams["font.family"] = ["SimHei", "WenQuanYi Micro Hei"]
12 plt.rcParams["axes.unicode_minus"] = False # 解决负号显示问题
13
14 warnings.filterwarnings('ignore')
15
16 # 1. 加载数据
17 try:
18     df = pd.read_excel('附件_处理后10.xlsx', engine='openpyxl')
19     print("数据加载成功!")
20     print(f"数据框形状: {df.shape}")
21     print(f"数据框列名: {df.columns.tolist()}")
22 except Exception as e:
23     print(f"文件加载失败: {str(e)}")
24     exit()
25
26 # 2. 数据探索和预处理
27 print("\n=== 数据探索 ===")
28 print(f"数据集总样本数: {len(df)}")
29
30 # 筛选出BMI在[28, 37]区间的数据
31 bmi_data = df[(df['孕妇BMI'] >= 28) & (df['孕妇BMI'] <= 37)].copy()
32
33 print(f"BMI在28-37之间的样本数: {len(bmi_data)}")
34
35 # 检查Y染色体浓度的分布
36 y_concentration = bmi_data['Y染色体浓度']
37 print(f"Y染色体浓度范围: [{y_concentration.min():.2f}, {y_concentration.max():.2f}]")
38 print(f"Y染色体浓度在3-5%之间的样本数: {len(bmi_data[(bmi_data['Y染色体浓度'] >= 3) & (bmi_data['Y染色体浓度'] <= 5)])}")
39
40 # 3. 创建BMI区间 (以0.01为步长)
41 bmi_data['BMI估计值'] = bmi_data['孕妇BMI'].round(2)
42
43 # 4. 改进的特征工程
44 features_list = []
45
46 # 遍历每一个唯一的BMI区间值
47 for bmi_int in np.around(np.arange(28.00, 37.01, 0.01), decimals=2):
48     # 筛选出当前BMI区间的数据
49     subset = bmi_data[bmi_data['BMI估计值'] == bmi_int]
50
51     if len(subset) == 0:
52         # 如果这个BMI区间完全没有数据, 跳过
53         continue
54
55     # 进一步筛选出Y染色体浓度在3%到5%之间的数据
56     subset_in_range = subset[(subset['Y染色体浓度'] >= 3) & (subset['Y染色体浓度'] <= 5)]
57
58     if len(subset_in_range) > 0:
59         # 计算该区间内Y浓度在3%-5%的样本比例
60         prop_in_range = len(subset_in_range) / len(subset)
61         # 计算该区间内Y浓度在3%-5%的样本的平均Y浓度
```

```
62     mean_y_concentration = subset_in_range['Y染色体浓度'].mean()
63     # 计算该区间内Y浓度在3%-5%的样本的Y浓度标准差
64     std_y_concentration = subset_in_range['Y染色体浓度'].std()
65     # 计算该区间内Y浓度在3%-5%的样本的平均孕周
66     mean_gestational_age = subset_in_range['检测孕周'].mean()
67
68     features_list.append({
69         'BMI估计值': bmi_int,
70         '样本总数': len(subset),
71         '目标范围样本数': len(subset_in_range),
72         '目标范围比例': prop_in_range,
73         'Y染色体浓度平均值': mean_y_concentration,
74         'Y染色体浓度标准差': std_y_concentration,
75         '检测孕周平均值': mean_gestational_age
76     })
77 else:
78     # 对于没有Y浓度在3-5%数据的区间，但仍然有样本，可以计算整体特征
79     prop_in_range = 0
80     mean_y_concentration = subset['Y染色体浓度'].mean()
81     std_y_concentration = subset['Y染色体浓度'].std()
82     mean_gestational_age = subset['检测孕周'].mean()
83
84     features_list.append({
85         'BMI估计值': bmi_int,
86         '样本总数': len(subset),
87         '目标范围样本数': 0,
88         '目标范围比例': 0,
89         'Y染色体浓度平均值': mean_y_concentration,
90         'Y染色体浓度标准差': std_y_concentration,
91         '检测孕周平均值': mean_gestational_age
```

```
92     })
93
94 # 检查是否计算出了特征
95 if not features_list:
96     print("错误：没有计算出任何特征。可能的原因:")
97     print("1. BMI 28-37区间内没有数据")
98     print("2. 所有BMI区间的Y染色体浓度都不在3-5%范围内")
99     print("请检查数据是否符合要求")
100     exit()
101
102 # 将特征列表转换为DataFrame
103 features_df = pd.DataFrame(features_list)
104 features_df.set_index('BMI估计值', inplace=True)
105
106 print(f"\n成功计算出 {len(features_df)} 个BMI区间的特征")
107
108 # 5. 处理缺失值
109 # 检查是否有缺失值
110 print(f"特征数据中的缺失值数量:")
111 print(features_df.isnull().sum())
112
113 # 使用每列的均值填充NaN值
114 for column in features_df.columns:
115     if features_df[column].isnull().sum() > 0:
116         col_mean = features_df[column].mean()
117         features_df[column].fillna(col_mean, inplace=True)
118         print(f"已用均值 {col_mean:.4f} 填充列 '{column}' 中的缺失值")
119
120 # 6. 选择用于聚类的特征
121 # 根据你的需求，选择能反映“Y浓度在3%到5%的孕期情况”的特征
```

```

122 feature_columns = ['目标范围比例', 'Y染色体浓度平均值', 'Y染色体浓度标准差', '检测孕周平均值']
123 print(f"\n用于聚类的特征: {feature_columns}")
124
125 # 7. 数据标准化[2](@ref)
126 scaler = StandardScaler()
127 X_scaled = scaler.fit_transform(features_df[feature_columns])
128
129 # 8. 确定合适的K值(簇数)
130 # 使用肘部法和轮廓系数法结合[1](@ref)
131 sse = []
132 silhouette_scores = []
133 k_range = range(2, 11) # 测试K从2到10
134
135 for k in k_range:
136     kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
137     kmeans.fit(X_scaled)
138     sse.append(kmeans.inertia_)
139
140     # 计算轮廓系数
141     if k > 1: # 轮廓系数需要至少2个簇
142         silhouette_avg = silhouette_score(X_scaled, kmeans.labels_)
143         silhouette_scores.append(silhouette_avg)
144     else:
145         silhouette_scores.append(0)
146
147 # 绘制肘部法图表和轮廓系数图表
148 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
149
150 # 肘部法图表
151 ax1.plot(k_range, sse, marker='o')
152
153 ax1.set_xlabel('簇数量 (K)')
154 ax1.set_ylabel('簇内平方误差和 (SSE)')
155 ax1.set_title('肘部法 - 最优K值选择')
156 ax1.grid(True)
157
158 # 轮廓系数图表
159 ax2.plot(k_range[1:], silhouette_scores[1:], marker='o', color='orange')
160 ax2.set_xlabel('簇数量 (K)')
161 ax2.set_ylabel('平均轮廓系数')
162 ax2.set_title('轮廓系数法 - 最优K值选择')
163 ax2.grid(True)
164
165 plt.tight_layout()
166 plt.show()
167
168 # 根据轮廓系数选择最优K值(选择轮廓系数最大的K)
169 if len(silhouette_scores) > 1:
170     optimal_k = np.argmax(silhouette_scores[1:]) + 2 # +2是因为从K=2开始
171     print(f"根据轮廓系数选择的最优K值为: {optimal_k}")
172 else:
173     # 如果只有少量数据, 使用较小的K值
174     optimal_k = min(3, len(features_df) // 5)
175     print(f"数据量较少, 使用K值为: {optimal_k}")
176
177 # 9. 使用最优K值进行K-Means聚类
178 kmeans_final = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
179 cluster_labels = kmeans_final.fit_predict(X_scaled)
180
181 # 将聚类结果添加回特征DataFrame
182 features_df['Cluster'] = cluster_labels

```

```

182
183 # 10. 评估聚类质量
184 if optimal_k > 1:
185     silhouette_avg = silhouette_score(X_scaled, cluster_labels)
186     print(f"平均轮廓系数 (Silhouette Score) for K={optimal_k}: {silhouette_avg:.4f}")
187 else:
188     print("K=1时无法计算轮廓系数")
189
190 # 11. 可视化结果 (使用PCA进行降维)
191 try:
192     pca = PCA(n_components=2)
193     X_pca = pca.fit_transform(X_scaled)
194
195     plt.figure(figsize=(10, 8))
196     scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=cluster_labels, cmap='viridis', alpha=0.6)
197     plt.xlabel('第一主成分')
198     plt.ylabel('第二主成分')
199     plt.title('K-means聚类结果 (PCA降维后)')
200     plt.colorbar(scatter, label='簇标签')
201
202     # 添加BMI值的标注 (只标注部分点, 避免过于拥挤)
203     for i, (idx, row) in enumerate(features_df.iterrows()):
204         if i % 10 == 0: # 每10个点标注一个
205             plt.annotate(f"{idx:.1f}", (X_pca[i, 0], X_pca[i, 1]), fontsize=8)
206
207     plt.show()
208 except Exception as e:
209     print(f"PCA可视化失败: {str(e)}")
210
211 # 12. 分析每个簇的特征中心 (均值)
212
213 cluster_characteristics = features_df.groupby('Cluster')[feature_columns].mean()
214 print("\n每个簇的特征平均值:")
215 print(cluster_characteristics)
216
217 # 13. 输出属于同一组的BMI区间
218 print("\n=== 根据聚类结果分组后的BMI区间 ===")
219 for cluster_id in range(optimal_k):
220     bmi_in_cluster = features_df.index[features_df['Cluster'] == cluster_id].values
221     cluster_samples = features_df[features_df['Cluster'] == cluster_id]['样本总数'].sum()
222     print(f"\n簇 {cluster_id} (包含 {len(bmi_in_cluster)} 个BMI区间, 总样本数: {cluster_samples}):")
223     print(f"BMI区间: {bmi_in_cluster}")
224
225     # 保存每个簇的BMI区间到文件
226     cluster_filename = f"cluster_{cluster_id}_bmi_ranges.csv"
227     pd.DataFrame(bmi_in_cluster, columns=['BMI区间']).to_csv(cluster_filename, index=False, encoding='utf-8-sig')
228     print(f"已保存到文件: {cluster_filename}")
229
230 # 14. 保存所有特征和聚类结果到文件
231 features_df.to_csv('聚类分析结果_详细.csv', encoding='utf-8-sig')
232 print(f"\n详细聚类分析结果已保存到 '聚类分析结果_详细.csv'")
233
234 # 15. 简要总结
235 print("\n=== 分析总结 ===")
236 print(f"共分析了 {len(features_df)} 个BMI区间")
237 print(f"最佳聚类数: {optimal_k}")
238 print("每个簇的简要特征:")
239
240 for cluster_id in range(optimal_k):
241     cluster_data = features_df[features_df['Cluster'] == cluster_id]
242     print(f"\n簇 {cluster_id}:")

```

```

242     print(f" - 包含区间数: {len(cluster_data)}")
243     print(f" - 平均目标范围比例: {cluster_data['目标范围比例'].mean():.3f}")
244     print(f" - 平均Y染色体浓度: {cluster_data['Y染色体浓度平均值'].mean():.3f}")
245     print(f" - 平均检测孕周: {cluster_data['检测孕周平均值'].mean():.3f}")
246
247 # 16. 扩展: 综合评估聚类质量与误差分析
248 print("\n=== 聚类质量综合评估与误差分析 ===\n")
249
250 # 16.1 计算多种内部评价指标
251 from sklearn.metrics import davies_bouldin_score, calinski_harabasz_score
252
253 # 计算戴维森-布尔丁指数 (DBI)[1,5](@ref)
254 # DBI值越小越好, 表示簇内距离小, 簇间距离大
255 dbi_score = davies_bouldin_score(X_scaled, cluster_labels)
256 print(f"戴维森-布尔丁指数 (DBI) for K={optimal_k}: {dbi_score:.4f}")
257
258 # 计算Calinski-Harabasz指数 (CHI)[4,5](@ref)
259 # CHI值越大越好, 表示簇间离散度与簇内离散度的比值高
260 chi_score = calinski_harabasz_score(X_scaled, cluster_labels)
261 print(f"Calinski-Harabasz指数 (CHI) for K={optimal_k}: {chi_score:.4f}")
262
263 # 结合轮廓系数进行解释[1,5](@ref)
264 if silhouette_avg > 0.7:
265     silhouette_eval = "聚类结构紧密且分离性好"
266 elif silhouette_avg > 0.5:
267     silhouette_eval = "聚类结构合理"
268 elif silhouette_avg > 0.25:
269     silhouette_eval = "聚类结构较弱, 可能存在重叠"
270 else:
271     silhouette_eval = "聚类结构可能无效"
272
273 print(f"轮廓系数评估: {silhouette_eval}")
274
275 # 16.2 评估聚类稳定性: 多次运行看结果一致性[1,6](@ref)
276 print("\n--- 聚类稳定性评估 ---")
277 n_iterations = 5 # 多次运行
278 consistent_count = 0
279 from sklearn.metrics import adjusted_rand_score
280
281 for i in range(n_iterations):
282     kmeans_temp = KMeans(n_clusters=optimal_k, n_init=1, init='random', random_state=i)
283     temp_labels = kmeans_temp.fit_predict(X_scaled)
284     # 使用调整兰德指数比较两次聚类结果的一致性[2](@ref)
285     ari = adjusted_rand_score(cluster_labels, temp_labels)
286     if ari > 0.9: # 认为ARI > 0.9表示结果高度一致
287         consistent_count += 1
288     print(f"第{i+1}次运行与初始结果的调整兰德指数 (ARI): {ari:.4f}")
289
290 stability_ratio = consistent_count / n_iterations
291 print(f"聚类稳定性: {stability_ratio*100:.1f}% 的运行次数与初始结果高度一致 (ARI > 0.9)")
292
293 # 16.3 分析簇内特征的变异 (衡量每个簇的同质性)[1](@ref)
294 print("\n--- 各簇内部特征变异系数分析 ---")
295 # 变异系数 = 标准差 / 均值, 衡量离散程度
296 for cluster_id in range(optimal_k):
297     cluster_data = features_df[features_df['Cluster'] == cluster_id]
298     print(f"\n簇 {cluster_id} 内部特征变异系数:")
299     for feature in feature_columns:
300         mean_val = cluster_data[feature].mean()
301         std_val = cluster_data[feature].std()
302         # 避免除零

```

```

302         if mean_val != 0:
303             cv = std_val / abs(mean_val)
304         else:
305             cv = np.nan
306         print(f"   {feature}: {cv:.4f} (均值: {mean_val:.4f}, 标准差: {std_val:.4f})")
307     # 计算该族的平均变异系数
308     cvs = []
309     for feature in feature_columns:
310         mean_val = cluster_data[feature].mean()
311         std_val = cluster_data[feature].std()
312         if mean_val != 0:
313             cvs.append(std_val / abs(mean_val))
314     avg_cv = np.mean(cvs) if cvs else np.nan
315     print(f"   该簇平均变异系数: {avg_cv:.4f}")
316
317 # 16.4 输出最终的聚类评估报告
318 print("\n=== 最终聚类评估报告 ===")
319 print(f"1. 最佳簇数 (K): {optimal_k}")
320 print(f"2. 轮廓系数: {silhouette_avg:.4f} - {silhouette_eval}")
321 print(f"3. 戴维森-布尔丁指数 (DBI): {dbi_score:.4f} (越小越好)")
322 print(f"4. Calinski-Harabasz指数 (CHI): {chi_score:.4f} (越大越好)")
323 print(f"5. 模型稳定性: {stability_ratio*100:.1f}% 的多次运行结果高度一致")
324 print("6. 建议:")
325 if silhouette_avg > 0.5 and dbi_score < 0.8 and stability_ratio > 0.7:
326     print("   聚类质量良好, 结果相对稳定可靠。")
327 elif silhouette_avg > 0.3 and dbi_score < 1.2 and stability_ratio > 0.5:
328     print("   聚类质量一般, 结果可供参考, 但应谨慎解读。")
329 else:
330     print("   聚类质量可能不理想, 建议检查数据特征、尝试其他聚类算法或调整参数。")

```

问题三代码

problem3ana. py

```

1  import pandas as pd
2  import seaborn as sns
3  import matplotlib.pyplot as plt
4  import matplotlib as mpl
5  import matplotlib.font_manager as fm
6
7
8  plt.rcParams['font.sans-serif'] = ['SimHei']
9  plt.rcParams['axes.unicode_minus'] = False
10
11
12 # 设置Seaborn样式
13 sns.set_style("whitegrid")
14 sns.set_context("notebook", font_scale=1.2)
15
16 # 数据读取
17 df = pd.read_excel('附件_处理后10.xlsx', sheet_name='男胎检测数据')
18
19 # 显示列名, 确保与代码中的列名匹配
20 print("数据列名:", df.columns.tolist())
21
22 # 创建图形
23 # 创建1行3列的子图, 设置整体图像大小
24 fig, axes = plt.subplots(1, 3, figsize=(18, 6))
25
26 # 1. 身高 vs Y染色体浓度散点图
27 sns.scatterplot(ax=axes[0], data=df, x='身高', y='Y染色体浓度',
28                 hue='年龄', palette='viridis', size='年龄',
29                 sizes=(20, 200), alpha=0.7)
30 axes[0].set_title('身高与Y染色体浓度的关系')
31 axes[0].set_xlabel('身高 (cm)')

```

```

32 axes[0].set_ylabel('Y染色体浓度')
33
34 # 2. 体重 vs Y染色体浓度散点图
35 sns.scatterplot(ax=axes[1], data=df, x='体重', y='Y染色体浓度',
36                 hue='年龄', palette='viridis', size='年龄',
37                 sizes=(20, 200), alpha=0.7)
38 axes[1].set_title('体重与Y染色体浓度的关系')
39 axes[1].set_xlabel('体重 (kg)')
40 axes[1].set_ylabel('Y染色体浓度')
41
42 # 3. 年龄 vs Y染色体浓度散点图
43 sns.scatterplot(ax=axes[2], data=df, x='年龄', y='Y染色体浓度',
44                 hue='身高', palette='coolwarm', size='体重',
45                 sizes=(20, 200), alpha=0.7)
46 axes[2].set_title('年龄与Y染色体浓度的关系')
47 axes[2].set_xlabel('年龄 (岁)')
48 axes[2].set_ylabel('Y染色体浓度')
49
50 # 调整布局并显示
51 plt.tight_layout()
52 plt.show()
53
54 plt.savefig('Y染色体浓度分析.png', dpi=300, bbox_inches='tight')
55 print('已保存图片')

```

problem3process.py

```

1  import pandas as pd
2  import numpy as np
3
4  # 读取数据
5  df = pd.read_excel('附件_处理后10.xlsx')
6
7  # 筛选出BMI在28到37之间的数据
8  filtered_df = df[(df['孕妇BMI'] >= 28) & (df['孕妇BMI'] <= 37)].copy()
9
10 # 创建区间: 从28到37, 步长为0.01
11 bins = np.arange(28, 37.01, 0.01)
12
13 # 使用pd.cut将BMI数据分配到各个区间
14 filtered_df['BMI区间'] = pd.cut(filtered_df['孕妇BMI'], bins=bins, include_lowest=True)
15
16 # 按区间分组并计算每个区间的Y染色体浓度平均值和标准差, 以及年龄、身高、体重、孕周的平均值
17 # 主要修改点: 在agg聚合函数中添加了 '孕周': 'mean'
18 result = filtered_df.groupby('BMI区间').agg({
19     'Y染色体浓度': ['mean', 'std'],
20     '年龄': 'mean',
21     '身高': 'mean',
22     '体重': 'mean',
23     '检测孕周': 'mean' # 新增对孕周列计算平均值
24 }).reset_index()
25
26 # 重命名列
27 result.columns = ['BMI区间', '平均Y染色体浓度', 'Y染色体浓度标准差', '平均年龄', '平均身高', '平均体重', '平均孕周']
28
29 # 计算每个区间的中点值 (作为代表性BMI值)
30 result['区间中点BMI'] = result['BMI区间'].apply(lambda x: x.mid)
31
32
33 # 将 '平均孕周' 加入最终结果的列中, 并调整到合适的位置
34 final_result = result[['区间中点BMI', '平均Y染色体浓度', 'Y染色体浓度标准差',
35                        '平均年龄', '平均身高', '平均体重', '平均孕周']]
36
37 # 导出到新的Excel文件
38 final_result.to_excel('BMI_多指标_分段平均值.xlsx', index=False, engine='openpyxl')
39
40 print("处理完成! 结果已保存到 'BMI_多指标_分段平均值.xlsx'")

```

problem3zhengshi.py

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.cluster import KMeans
5 from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score, adjusted_rand_score
6 from sklearn.decomposition import PCA
7 import matplotlib.pyplot as plt
8 from scipy import stats
9 import warnings
10
11 # 设置中文显示
12 plt.rcParams["font.family"] = ["SimHei", "WenQuanYi Micro Hei"]
13 plt.rcParams["axes.unicode_minus"] = False # 解决负号显示问题
14 warnings.filterwarnings('ignore')
15
16 # 1. 数据加载与预处理
17 print("=== 数据加载与预处理 ===")
18 try:
19     df = pd.read_excel('BMI_多指标_分段平均值.xlsx')
20     print("数据加载成功!")
21     print(f"数据框形状: {df.shape}")
22     print(f"数据框列名: {df.columns.tolist()}")
23 except Exception as e:
24     print(f"文件加载失败: {str(e)}")
25     exit()
26
27 # 2. 数据探索
28 print("\n=== 数据探索 ===")
29 print(f"数据集总样本数: {len(df)}")
30
31 # 过滤数据: 只保留Y染色体浓度在3%到5%之间的样本
32
33 df_filtered = df[(df['平均Y染色体浓度'] >= 0.03) & (df['平均Y染色体浓度'] <= 0.05)].copy()
34 print(f"Y染色体浓度在3%-5%之间的样本数: {len(df_filtered)}")
35 print(f"Y染色体浓度范围: [{df_filtered['平均Y染色体浓度'].min():.4f}, {df_filtered['平均Y染色体浓度'].max():.4f}]")
36
37 # 显示基本统计信息
38 print("\n各特征基本统计信息:")
39 print(df_filtered[['平均年龄', '平均身高', '平均体重', '区间中点BMI', '平均孕周']].describe())
40
41 # 3. 特征选择与加权
42 print("\n=== 特征工程 ===")
43 features = df_filtered[['平均年龄', '平均身高', '平均体重', '区间中点BMI']].copy()
44 weights = np.array([0.1, 0.1, 0.1, 0.7]) # 对应特征的权重
45 features_weighted = features * weights # 应用权重
46 print("已对特征进行加权处理: 年龄(10%), 身高(10%), 体重(10%), BMI(70%)")
47
48 # 4. 数据标准化
49 scaler = StandardScaler()
50 features_scaled = scaler.fit_transform(features_weighted)
51 print("已完成数据标准化处理")
52
53 # 5. 确定最佳聚类数量K (使用肘部法和轮廓系数法)
54 print("\n=== 确定最佳聚类数量 ===")
55 sse = []
56 silhouette_scores = []
57 k_range = range(2, 11) # 测试K从2到10
58
59 for k in k_range:
60     kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
61     kmeans.fit(features_scaled)
62     sse.append(kmeans.inertia_)
```

```

62
63     if k > 1: # 轮廓系数需要至少2个簇
64         silhouette_avg = silhouette_score(features_scaled, kmeans.labels_)
65         silhouette_scores.append(silhouette_avg)
66     else:
67         silhouette_scores.append(0)
68
69 # 绘制评估图表
70 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))
71
72 # 肘部法图表
73 ax1.plot(k_range, sse, marker='o')
74 ax1.set_xlabel('簇数量 (K)')
75 ax1.set_ylabel('簇内平方误差和 (SSE)')
76 ax1.set_title('肘部法 - 最优K值选择')
77 ax1.grid(True)
78
79 # 轮廓系数图表
80 ax2.plot(k_range, silhouette_scores, marker='o', color='orange')
81 ax2.set_xlabel('簇数量 (K)')
82 ax2.set_ylabel('平均轮廓系数')
83 ax2.set_title('轮廓系数法 - 最优K值选择')
84 ax2.grid(True)
85
86 plt.tight_layout()
87 plt.show()
88
89 # 根据轮廓系数选择最优K值
90 optimal_k = k_range[np.argmax(silhouette_scores)]
91 print(f"根据轮廓系数选择的最优K值为: {optimal_k}")
92
93 # 6. 使用K-Means进行聚类
94 print("\n=== 执行K-Means聚类 ===")
95 kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
96 clusters = kmeans.fit_predict(features_scaled)
97 df_filtered['Cluster'] = clusters
98 print(f"已完成K-Means聚类, 共分为{optimal_k}个簇")
99
100 # 7. 分析每个簇的特征
101 print("\n=== 簇特征分析 ===")
102 # 计算原始尺度下的簇中心 (去加权)
103 cluster_centers_original = scaler.inverse_transform(kmeans.cluster_centers_) / weights
104 cluster_characteristics = pd.DataFrame(cluster_centers_original, columns=features.columns)
105 cluster_characteristics['样本数量'] = np.bincount(clusters)
106 cluster_characteristics['占比'] = cluster_characteristics['样本数量'] / len(df_filtered) * 100
107
108 print("各簇的特征中心 (原始尺度, 已去加权):")
109 print(cluster_characteristics)
110
111 # 8. 寻找每个簇最适合的NIPT检测时间点
112 print("\n=== 推荐NIPT检测时间点 ===")
113 recommended_times = {}
114 for cluster_id in range(optimal_k):
115     cluster_data = df_filtered[df_filtered['Cluster'] == cluster_id]
116     recommended_time = cluster_data['平均孕周'].median()
117     recommended_times[f'簇_{cluster_id}'] = recommended_time
118     print(f"簇 {cluster_id}: 推荐孕周 = {recommended_time:.2f}")
119
120 # 9. 聚类质量评估
121 print("\n=== 聚类质量评估 ===")

```

```

122 silhouette_avg = silhouette_score(features_scaled, clusters)
123 dbi_score = davies_bouldin_score(features_scaled, clusters)
124 chi_score = calinski_harabasz_score(features_scaled, clusters)
125
126 print(f"平均轮廓系数: {silhouette_avg:.4f}")
127 print(f"戴维森-布尔丁指数 (DBI): {dbi_score:.4f} (越小越好)")
128 print(f"Calinski-Harabasz指数 (CHI): {chi_score:.4f} (越大越好)")
129
130 # 评估轮廓系数
131 if silhouette_avg > 0.7:
132     silhouette_eval = "聚类结构紧密且分离性好"
133 elif silhouette_avg > 0.5:
134     silhouette_eval = "聚类结构合理"
135 elif silhouette_avg > 0.25:
136     silhouette_eval = "聚类结构较弱, 可能存在重叠"
137 else:
138     silhouette_eval = "聚类结构可能无效"
139 print(f"轮廓系数评估: {silhouette_eval}")
140
141 # 10. 稳定性分析
142 print("\n=== 聚类稳定性分析 ===")
143 n_iterations = 10
144 ari_scores = []
145 reference_labels = clusters
146
147 for i in range(n_iterations):
148     kmeans_temp = KMeans(n_clusters=optimal_k, random_state=i * 100, n_init=10)
149     temp_labels = kmeans_temp.fit_predict(features_scaled)
150     ari = adjusted_rand_score(reference_labels, temp_labels)
151     ari_scores.append(ari)
152
153     print(f"第{i + 1}次运行与参考结果的调整兰德指数(ARI): {ari:.4f}")
154
155 avg_ari = np.mean(ari_scores)
156 print(f"平均调整兰德指数(ARI): {avg_ari:.4f}")
157
158 if avg_ari > 0.9:
159     print("聚类结果稳定性: 高 (ARI > 0.9)")
160 elif avg_ari > 0.7:
161     print("聚类结果稳定性: 中等 (ARI > 0.7)")
162 else:
163     print("聚类结果稳定性: 低 (ARI ≤ 0.7)")
164
165 # 11. 可视化结果
166 print("\n=== 结果可视化 ===")
167 # PCA降维可视化
168 pca = PCA(n_components=2)
169 features_pca = pca.fit_transform(features_scaled)
170
171 plt.figure(figsize=(10, 8))
172 scatter = plt.scatter(features_pca[:, 0], features_pca[:, 1], c=clusters, cmap='viridis', alpha=0.7)
173 plt.colorbar(scatter, label='簇')
174 plt.xlabel('第一主成分')
175 plt.ylabel('第二主成分')
176 plt.title('K-Means聚类结果 (PCA降维可视化)')
177
178 # 标记簇中心
179 for cluster_id in range(optimal_k):
180     cluster_center_pca = pca.transform(kmeans.cluster_centers_[cluster_id].reshape(1, -1))
181     plt.scatter(cluster_center_pca[:, 0], cluster_center_pca[:, 1], c='red', marker='X', s=200, linewidths=3)
182 plt.grid(True)

```

```

182 plt.show()
183
184 # 12. 输出最终分组结果
185 print("\n=== 最终分组结果 ===")
186 print("原始数据与聚类标签 (前10行):")
187 print(
188     df_filtered[['平均年龄', '平均身高', '平均体重', '区间中点BMI', '平均Y染色体浓度', '平均孕周', 'Cluster']].head(10))
189
190 # 13. 保存结果
191 print("\n=== 结果保存 ===")
192 df_filtered.to_csv('聚类分析结果_详细.csv', encoding='utf-8-sig')
193 print("详细聚类分析结果已保存到 '聚类分析结果_详细.csv'")
194
195 # 14. 分析总结
196 print("\n=== 分析总结 ===")
197 print(f"共分析了 {len(df_filtered)} 个样本")
198 print(f"最佳聚类数: {optimal_k}")
199 print("各簇样本分布:")
200 for cluster_id in range(optimal_k):
201     cluster_size = len(df_filtered[df_filtered['Cluster'] == cluster_id])
202     print(f"簇 {cluster_id}: {cluster_size} 个样本 ({cluster_size / len(df_filtered) * 100:.1f}%)")
203
204 print("\n建议:")
205 if silhouette_avg > 0.5 and dbi_score < 0.8 and avg_ari > 0.7:
206     print("聚类质量良好, 结果相对稳定可靠, 可以信赖分组结果。")
207 elif silhouette_avg > 0.3 and dbi_score < 1.2 and avg_ari > 0.5:
208     print("聚类质量一般, 结果可供参考, 但应结合业务背景谨慎解读。")
209 else:
210     print("聚类质量可能不理想, 建议重新评估特征选择、权重分配或尝试其他聚类算法。")

```

问题四代码

problem4(1).py

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.impute import SimpleImputer
6 from sklearn.pipeline import Pipeline
7 from sklearn.covariance import EllipticEnvelope
8 from sklearn.ensemble import IsolationForest, RandomForestClassifier
9 from sklearn.neighbors import LocalOutlierFactor
10 from sklearn.model_selection import train_test_split
11
12 # 1. 导入数据集
13 file_path = '附件_处理后10.xlsx'
14
15 try:
16     excel_file = pd.ExcelFile(file_path)
17     df = excel_file.parse('女胎检测数据')
18     print(f"成功导入数据, 共 {df.shape[0]} 行, {df.shape[1]} 列")
19 except FileNotFoundError:
20     print(f"错误: 找不到文件 '{file_path}', 请检查文件路径是否正确")
21     exit()
22 except Exception as e:
23     print(f"读取数据时发生错误: {str(e)}")
24     exit()
25
26
27 # 2. 数据预处理 - 处理染色体非整倍体特征 (支持复合异常)
28 def parse_chromosome_abnormal(code):
29     """
30     解析染色体非整倍体编码:
31     code: 原始编码 (如0=无异常, 1=13号, 2=18号, 3=21号, 12=13+18号, 13=13+21号, 23=18+21号, 123=三者均异常)

```

```

32     返回: 13号、18号、21号染色体是否异常的二元值
33     """
34     code_str = str(int(code)) # 转为字符串便于逐位判断
35     return {
36         '13号染色体非整倍体': 1 if '1' in code_str else 0,
37         '18号染色体非整倍体': 1 if '2' in code_str else 0,
38         '21号染色体非整倍体': 1 if '3' in code_str else 0
39     }
40
41
42 # 应用解析函数, 生成正确的特征
43 abnormal_features = df['染色体的非整倍体'].apply(parse_chromosome_abnormal).apply(pd.Series)
44 df = pd.concat([df, abnormal_features], axis=1)
45
46 # 验证复合异常样本的处理情况
47 print("\n复合染色体非整倍体样本处理验证: ")
48 compound_samples = df[df['染色体的非整倍体'].isin([12, 13, 23, 123]).head(5)
49 print(compound_samples[['染色体的非整倍体', '13号染色体非整倍体', '18号染色体非整倍体', '21号染色体非整倍体']])
50
51 # 3. 定义特征集
52 features = [
53     '孕妇BMI', 'X染色体浓度',
54     '13号染色体的Z值', '18号染色体的Z值', '21号染色体的Z值',
55     '13号染色体的GC含量', '18号染色体的GC含量', '21号染色体的GC含量',
56     '被过滤掉读数的比例',
57     '13号染色体非整倍体', '18号染色体非整倍体', '21号染色体非整倍体'
58 ]
59
60 X = df[features]
61
62
63 # 4. 数据预处理管道
64 preprocessor = Pipeline([
65     ('imputer', SimpleImputer(strategy='median')), # 使用中位数填充缺失值, 更稳健
66     ('scaler', StandardScaler()) # 标准化特征
67 ])
68
69 # 预处理数据
70 X_processed = preprocessor.fit_transform(X)
71
72 # 5. 配置中文显示 (解决方框问题)
73 plt.rcParams['font.sans-serif'] = ['SimHei', 'WenQuanYi Micro Hei', 'Heiti TC'] # 支持多种中文字体
74 plt.rcParams['axes.unicode_minus'] = False # 解决负号显示问题
75
76 # 6. 异常检测模型
77 # 初始化多个异常检测模型 (多角度检测异常)
78 models = {
79     # 基于高斯分布的异常检测
80     '椭圆包络': EllipticEnvelope(contamination=0.05, random_state=42),
81     # 基于隔离森林的异常检测
82     '隔离森林': IsolationForest(contamination=0.05, n_estimators=100, random_state=42),
83     # 基于局部离群因子的异常检测
84     '局部离群因子': LocalOutlierFactor(n_neighbors=20, contamination=0.05, novelty=True)
85 }
86
87 # 训练模型并预测异常分数
88 results = df.copy() # 保留原始数据用于结果分析
89
90 for name, model in models.items():
91     # 训练模型 (使用所有健康样本)
92     model.fit(X_processed)

```

```

92
93     # 预测异常标签 (-1表示异常, 1表示正常)
94     results[f'{name}_异常标签'] = model.predict(X_processed)
95
96     # 获取异常分数 (分数越低越可能是异常)
97     if name == '局部离群因子':
98         # 局部离群因子的决策函数返回负的异常分数, 取相反数使其与其他模型一致
99         results[f'{name}_异常分数'] = -model.decision_function(X_processed)
100     else:
101         results[f'{name}_异常分数'] = model.decision_function(X_processed)
102
103
104 # 7. 综合异常判定
105 def calculate_anomaly_probability(row):
106     """综合多个模型结果计算异常概率 (0-100%)"""
107     anomaly_flags = [
108         row['椭圆包络_异常标签'] == -1,
109         row['隔离森林_异常标签'] == -1,
110         row['局部离群因子_异常标签'] == -1
111     ]
112     # 计算被多少模型判定为异常的比例
113     return sum(anomaly_flags) / len(anomaly_flags) * 100
114
115
116 results['综合异常概率(%)'] = results.apply(calculate_anomaly_probability, axis=1)
117
118 # 设置判定阈值: 当2/3以上模型判定为异常时, 建议进一步检查
119 results['是否建议进一步检查'] = results['综合异常概率(%)'] > 66.7
120
121 # 8. 数据格式化处理

```

```

122 # ① 有小数的列的所有数据保留四位小数, 若未满则补零
123 float_columns = results.select_dtypes(include=['float']).columns.tolist()
124 for col in float_columns:
125     if col != '综合异常概率(%)':
126         results[col] = results[col].apply(lambda x: f'{x:.4f}')
127
128 # ② 异常概率列中所有数据保留两位小数, 若未满则补零
129 results['综合异常概率(%)'] = results['综合异常概率(%)'].apply(lambda x: f'{x:.2f}')
130
131 # ③ 处理空列和题头
132 unnamed_cols = [col for col in results.columns if 'unnamed' in col.lower()]
133 results = results.drop(unnamed_cols, axis=1)
134
135 # 9. 结果分析与可视化
136 # 显示最可能异常的前10个样本
137 print("\n综合异常概率最高的前10个样本:")
138 top_anomalies = results.sort_values('综合异常概率(%)', ascending=False).head(10)
139 print(top_anomalies[['孕代码', '检测孕周', '染色体的非整倍体', '综合异常概率(%)']])
140
141 # 可视化异常分数分布 (并保存为图片)
142 plt.figure(figsize=(15, 5))
143 for i, name in enumerate(models.keys()):
144     plt.subplot(1, 3, i + 1)
145     try:
146         n, bins, patches = plt.hist(results[f'{name}_异常分数'].astype(float), bins=30, alpha=0.7, color='#2a9d8f')
147         threshold = np.median(results[f'{name}_异常分数'].astype(float))
148         plt.axvline(x=threshold, color='red', linestyle='--', label=f'异常阈值: {threshold:.2f}')
149     except:
150         pass
151     plt.title(f'{name}异常分数分布', fontsize=12)

```

```

152     plt.xlabel('异常分数', fontsize=10)
153     plt.ylabel('样本数', fontsize=10)
154     plt.grid(axis='y', alpha=0.3)
155     plt.legend()
156
157 plt.tight_layout()
158 image_path = '异常分数分布图.png'
159 plt.savefig(image_path, dpi=300, bbox_inches='tight')
160 plt.close()
161 print(f"\n异常分数分布图已保存至: {image_path}")
162
163 # 10. 特征重要性分析
164 # 训练随机森林模型评估特征重要性
165 X_train, X_test, y_train, y_test = train_test_split(
166     X_processed,
167     results['是否建议进一步检查'].astype(int),
168     test_size=0.2,
169     random_state=42
170 )
171
172 rf = RandomForestClassifier(n_estimators=100, random_state=42)
173 rf.fit(X_train, y_train)
174
175 # 显示特征重要性
176 importance = pd.DataFrame({
177     '特征': features,
178     '重要性': rf.feature_importances_
179 }).sort_values('重要性', ascending=False)
180
181 print("\n对异常判定影响最大的特征:")
182
183 print(importance)
184
185 # 11. 绘制带图例的饼状图（优化布局避免拥挤）
186 plt.figure(figsize=(14, 10)) # 扩大画布尺寸
187 filtered_importance = importance[importance['重要性'] > 0.01] # 过滤微小特征
188
189 # 生成彩色系
190 colors = plt.cm.Set3(np.linspace(0, 1, len(filtered_importance)))
191
192 # 绘制饼状图，不显示标签，后续在图例中展示
193 wedges, texts, autotexts = plt.pie(
194     filtered_importance['重要性'],
195     autopct='%1.1f%%',
196     startangle=90,
197     colors=colors
198 )
199
200 # 调整百分比文本样式
201 for autotext in autotexts:
202     autotext.set_fontsize(10)
203     autotext.set_color('white')
204     autotext.set_weight('bold')
205
206 # 添加图例，设置位置与示例图对应
207 plt.legend(
208     filtered_importance['特征'],
209     title='特征名称',
210     loc='center left',
211     bbox_to_anchor=(1, 0.5),
212     fontsize=10,

```

```

212     title_fontsize=12
213 )
214
215 plt.title('特征重要性占比饼状图', fontsize=16, pad=20)
216 plt.axis('equal')
217 plt.tight_layout()
218 plt.subplots_adjust(right=0.75) # 调整右侧边距, 为图例留出空间
219
220 # 保存图片
221 pie_image_path = '特征重要性彩色饼状图.png'
222 plt.savefig(pie_image_path, dpi=300, bbox_inches='tight', pad_inches=0.5)
223 plt.close()
224 print(f"\n特征重要性彩色饼状图已保存至: {pie_image_path}")
225
226 # 12. 保存完整结果
227 output_path = '女胎异常检测结果_完整版.xlsx'
228 results.to_excel(output_path, index=False)
229 print(f"\n所有检测结果已保存至: {output_path}")

```

problem4(2).py

```

1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.impute import SimpleImputer
6 from sklearn.pipeline import Pipeline
7 from sklearn.covariance import EllipticEnvelope
8 from sklearn.ensemble import IsolationForest, RandomForestClassifier
9 from sklearn.neighbors import LocalOutlierFactor
10 from sklearn.model_selection import train_test_split
11 from sklearn.metrics import accuracy_score, precision_score, recall_score
12
13 # 1. 导入数据集
14 file_path = '附件_处理后10.xlsx'
15
16 try:
17     excel_file = pd.ExcelFile(file_path)
18     df = excel_file.parse('女胎检测数据')
19     print(f"成功导入数据, 共 {df.shape[0]} 行, {df.shape[1]} 列")
20 except FileNotFoundError:
21     print(f"错误: 找不到文件 '{file_path}'")
22     exit()
23 except Exception as e:
24     print(f"读取数据错误: {str(e)}")
25     exit()
26
27
28 # 2. 数据预处理 - 染色体异常解析
29 def parse_chromosome_abnormal(code):
30     code_str = str(int(code))
31     return {

```

```

32         '13号染色体非整倍体': 1 if '1' in code_str else 0,
33         '18号染色体非整倍体': 1 if '2' in code_str else 0,
34         '21号染色体非整倍体': 1 if '3' in code_str else 0
35     }
36
37
38 abnormal_features = df['染色体的非整倍体'].apply(parse_chromosome_abnormal).apply(pd.Series)
39 df = pd.concat([df, abnormal_features], axis=1)
40
41 # 3. 特征工程 - 强化21号染色体特征（基于原有特征）
42 core_z_features = ['13号染色体的Z值', '18号染色体的Z值', '21号染色体的Z值']
43 aux_features = [
44     '孕妇BMI', 'X染色体浓度',
45     '13号染色体的GC含量', '18号染色体的GC含量', '21号染色体的GC含量',
46     '被过滤掉读段数的比例',
47     '13号染色体非整倍体', '18号染色体非整倍体', '21号染色体非整倍体'
48 ]
49 features = core_z_features + aux_features
50
51 X = df[features].copy()
52
53 # 临床优先级加权 - 提高21号染色体Z值权重
54 clinical_weights = {
55     '21号染色体的Z值': 15.0,
56     '18号染色体的Z值': 6.0,
57     '13号染色体的Z值': 5.0,
58     '孕妇BMI': 0.5,
59     'X染色体浓度': 0.5,
60     '13号染色体的GC含量': 0.3,
61     '18号染色体的GC含量': 0.3,
62     '21号染色体的GC含量': 0.3,
63     '被过滤掉读段数的比例': 0.3,
64     '13号染色体非整倍体': 0.4,
65     '18号染色体非整倍体': 0.4,
66     '21号染色体非整倍体': 0.4
67 }
68
69 X_weighted = X.copy()
70 for feature, weight in clinical_weights.items():
71     X_weighted[feature] = X_weighted[feature] * weight
72
73 # 4. 数据预处理管道
74 preprocessor = Pipeline([
75     ('imputer', SimpleImputer(strategy='median')),
76     ('scaler', StandardScaler())
77 ])
78
79 X_processed = preprocessor.fit_transform(X_weighted)
80 X_processed_df = pd.DataFrame(X_processed, columns=features, index=X.index)
81
82 # 5. 异常检测模型
83 models = {
84     '椭圆包络': EllipticEnvelope(contamination=0.05, random_state=42),
85     '隔离森林': IsolationForest(
86         contamination=0.05,
87         n_estimators=100,
88         max_samples='auto',
89         random_state=42
90     ),
91     '局部离群因子': LocalOutlierFactor(

```

```

92         n_neighbors=15,
93         contamination=0.05,
94         novelty=True
95     )
96 }
97
98 results = df.copy()
99 for name, model in models.items():
100     model.fit(X_processed)
101     results[f'{name}_异常标签'] = model.predict(X_processed)
102     if name == '局部离群因子':
103         results[f'{name}_异常分数'] = -model.decision_function(X_processed)
104     else:
105         results[f'{name}_异常分数'] = model.decision_function(X_processed)
106
107
108 # 6. 综合异常判定 - 21号染色体优先
109 def calculate_anomaly_probability(row):
110     # 21号染色体Z值优先判定
111     if abs(row['21号染色体的Z值']) > 2.0:
112         return 100.0
113     elif 1.5 < abs(row['21号染色体的Z值']) <= 2.0:
114         return 95.0
115     elif abs(row['18号染色体的Z值']) > 3.0:
116         return 90.0
117     elif abs(row['13号染色体的Z值']) > 3.0:
118         return 80.0
119     else:
120         anomaly_flags = [
121             row['椭圆包络_异常标签'] == -1,
122             row['隔离森林_异常标签'] == -1,
123             row['局部离群因子_异常标签'] == -1
124         ]
125         base_prob = sum(anomaly_flags) / len(anomaly_flags) * 100
126         return base_prob
127
128
129 results['综合异常概率(%)'] = results.apply(calculate_anomaly_probability, axis=1)
130 results['是否建议进一步检查'] = results['综合异常概率(%)'] > 60.0
131
132 # 7. 数据格式化处理
133 float_columns = results.select_dtypes(include=['float']).columns.tolist()
134 for col in float_columns:
135     if col != '综合异常概率(%)':
136         results[col] = results[col].apply(lambda x: f'{x:.4f}')
137 results['综合异常概率(%)'] = results['综合异常概率(%)'].apply(lambda x: f'{x:.2f}')
138 unnamed_cols = [col for col in results.columns if 'unnamed' in col.lower()]
139 results = results.drop(unnamed_cols, axis=1)
140
141 # 8. 模型训练与评估 + 优化饼状图
142 y = results['是否建议进一步检查'].astype(int)
143 # 分层抽样确保21号染色体相关样本分布合理
144 stratify_col = df['21号染色体非整倍体'].astype(str) + "_" + y.astype(str)
145 X_train, X_test, y_train, y_test = train_test_split(
146     X_processed_df,
147     y,
148     test_size=0.2,
149     random_state=42,
150     stratify=stratify_col
151 )

```

```

152
153 # 调整随机森林参数，增强对21号染色体特征的学习
154 rf = RandomForestClassifier(
155     n_estimators=200, # 增加树的数量，提升学习能力
156     max_depth=6, # 较浅深度，防止过拟合其他特征
157     min_samples_split=5, # 降低分裂阈值，更容易基于21号特征分裂
158     min_samples_leaf=1,
159     class_weight='balanced',
160     random_state=42,
161     max_features='sqrt' # 限制每次分裂的特征数量，增加21号特征被选中概率
162 )
163 rf.fit(X_train, y_train)
164
165 # 特征重要性
166 importance = pd.DataFrame({
167     '特征': features,
168     '重要性': rf.feature_importances_
169 }).sort_values('重要性', ascending=False)
170
171 print("\n特征重要性排名（临床优先级：21号Z值 > 18号Z值 > 13号Z值）:")
172 print(importance)
173
174 # 解决中文显示问题
175 plt.rcParams['font.sans-serif'] = ['SimHei']
176 plt.rcParams['axes.unicode_minus'] = False
177
178 # 定义颜色列表
179 colors = [
180     '#8FD3E8', '#FFFACD', '#C9CCD5', '#E9D8A6', '#BBE1FA',
181     '#94B49F', '#B5E48C', '#FFD7BA', '#A3C4F3', '#F7CACA',

```

```

182     '#C4F1C4', '#FDE74C'
183 ]
184
185 # 绘制饼状图（优化布局）
186 plt.figure(figsize=(14, 8)) # 增大画布宽度
187 filtered_importance = importance[importance['重要性'] > 0.01]
188
189 # 提取特征名称和对应颜色，用于图例
190 feature_names = filtered_importance['特征'].tolist()
191 feature_colors = colors[:len(feature_names)]
192
193 # 绘制饼图，不显示标签（标签将在图例中显示）
194 wedges, texts, autotexts = plt.pie(
195     filtered_importance['重要性'],
196     autopct='%1.1f%%', # 只显示百分比
197     startangle=90,
198     colors=feature_colors,
199     pctdistance=0.85, # 百分比位置
200     wedgeprops=dict(width=0.7) # 使用稍窄的饼图，增加空间
201 )
202
203 # 调整百分比文本
204 for autotext in autotexts:
205     autotext.set_fontsize(10)
206     autotext.set_color('white')
207     autotext.set_weight('bold')
208
209 # 添加图例，放在右侧，解决标签拥挤问题
210 plt.legend(
211     feature_names,

```

```
212     title="特征名称",
213     loc="center left",
214     bbox_to_anchor=(1, 0.5), # 图例位置在图表右侧中间
215     fontsize=10,
216     title_fontsize=12
217 )
218
219 plt.title('特征重要性占比饼状图', fontsize=14, pad=20)
220 plt.axis('equal')
221 plt.tight_layout()
222 plt.subplots_adjust(right=0.75) # 调整右侧边距，为图例留出空间
223 plt.show()
224
225 # 模型性能
226 y_pred = rf.predict(X_test)
227 print("\n模型整体性能评估:")
228 print(f"准确率: {accuracy_score(y_test, y_pred):.4f}")
229 print(f"精确率: {precision_score(y_test, y_pred):.4f}")
230 print(f"召回率: {recall_score(y_test, y_pred):.4f}")
231
232 # 9. 保存结果
233 output_path = '女胎异常检测结果_21号强化版.xlsx'
234 results.to_excel(output_path, index=False)
235 print(f"\n结果已保存至: {output_path}")
```
